

```

import cv2
import numpy as np
from google.colab.patches import cv2_imshow # Import cv2_imshow
from IPython.display import display, HTML # Import display for better error

# Open the video file
video_path = '/beach.mp4' # Use the provided video file
cap = cv2.VideoCapture(video_path)

if not cap.isOpened():
    display(HTML(f"<p style='color:red;'>Error: Could not open video file .
    cap.release()
else:
    # Original animation context assumed 640x480. We'll resize video frame:
    target_width, target_height = 640, 480

    # Get original video properties for logging/information
    original_video_width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
    original_video_height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
    fps = cap.get(cv2.CAP_PROP_FPS)
    total_frames_in_video = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

    display(HTML(f"<p>Processing video: <b>{video_path}</b></p>"))
    display(HTML(f"<p>Original video resolution: {original_video_width}x{original_video_height}</p>"))
    display(HTML(f"<p>Resizing frames to: {target_width}x{target_height} frames</p>"))

    # Process for 1 second (or less if video is shorter)
    duration_to_process_seconds = 1
    frames_to_process = min(total_frames_in_video, int(fps * duration_to_process_seconds))

    display(HTML(f"<p>Attempting to process <b>{frames_to_process}</b> frames</p>"))

    # Ball position and speed (relative to target_width, target_height)
    x, y = 100, 100 # Initial position
    dx, dy = 5, 4 # Speed

```

```

for i in range(frames_to_process):
    ret, frame = cap.read()
    if not ret:
        display(HTML(f"<p style='color:orange;'>Warning: Failed to read frame {i+1}"))
        break

    # Resize the video frame to the target resolution
    frame_processed = cv2.resize(frame, (target_width, target_height))

    # Draw moving ball
    cv2.circle(frame_processed, (x, y), 30, (0, 255, 255), -1)

    # Draw moving rectangle
    cv2.rectangle(frame_processed, (200, 150), (350, 280), (255, 0, 0), 2)

    # Draw moving line
    # The line's vertical position depends on the frame index and wrap around
    cv2.line(frame_processed, (0, i % target_height), (target_width, (i % target_height)), (0, 0, 255), 2)

    # Display frame number
    cv2.putText(frame_processed, f"Frame: {i+1}", (20, 40),
                cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255), 2)

    # Update ball position
    x += dx
    y += dy

    # Ball bouncing logic - assuming target_width x target_height bounds
    if x >= target_width-30 or x <= 30:
        dx = -dx
    if y >= target_height-30 or y <= 30:
        dy = -dy

    # Source points for perspective transform
    # These are fixed points within the target_width x target_height frame
    src = np.float32([
        [100, 100],
        [500, 100],
        [500, 500],
        [100, 500]
    ])

```

```
[510,100],
[100,380],
[540,380]
])

# Destination points for perspective transform
dst = np.float32([
    [50,50],
    [590,100],
    [120,430],
    [520,380]
])

# Perspective Transformation
M = cv2.getPerspectiveTransform(src, dst)
output = cv2.warpPerspective(frame_processed, M, (target_width, ta

# Display both the augmented original frame and the perspective tr
cv2.imshow(frame_processed)
cv2.imshow(output)

# Release the video capture object
cap.release()
display(HTML(f"<p>Video processing finished for <b>{video_path}</b>.</p>"))
```

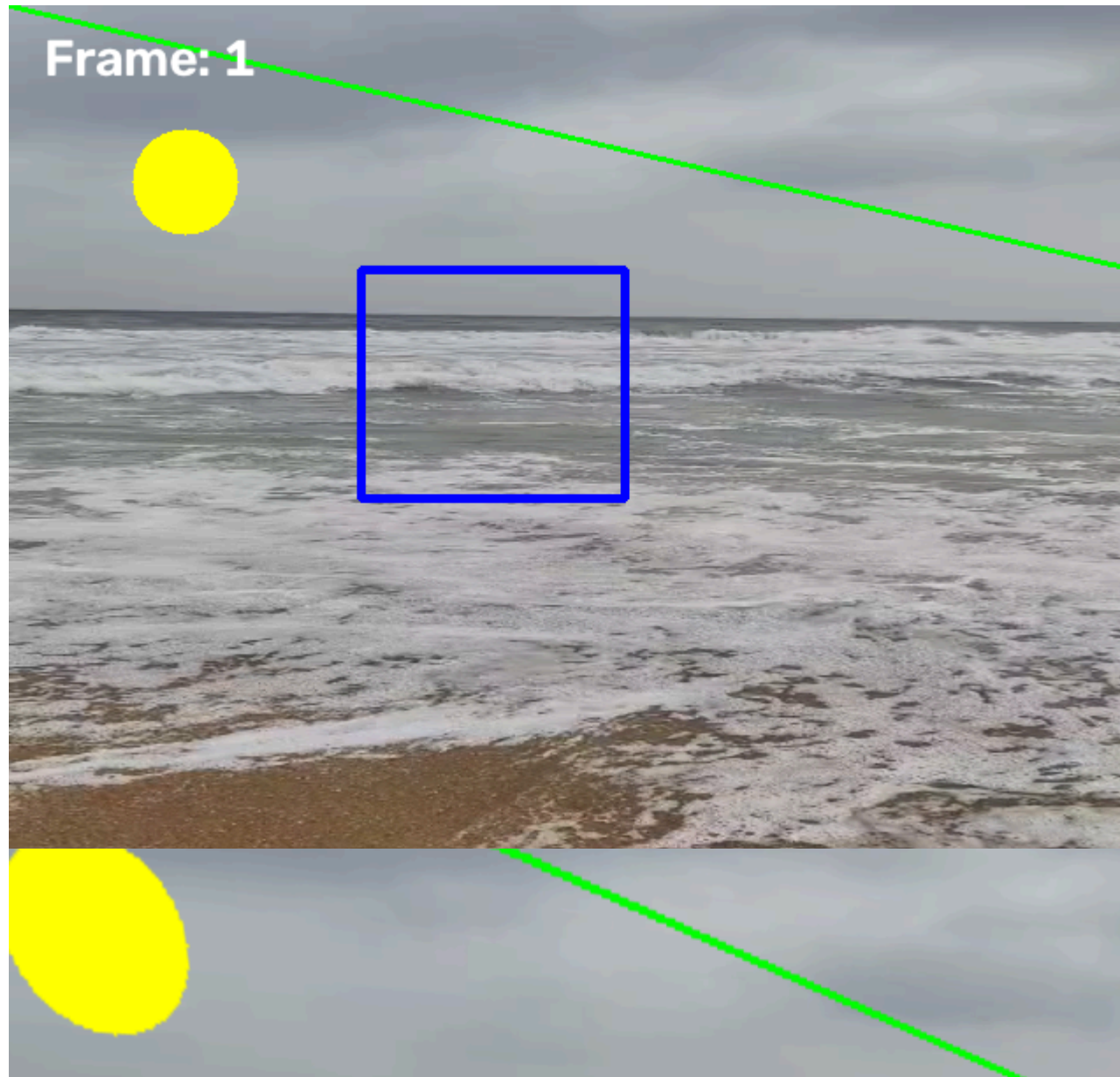


Processing video: **/beach.mp4**

Original video resolution: 478x850, FPS: 30.03, Total frames: 384

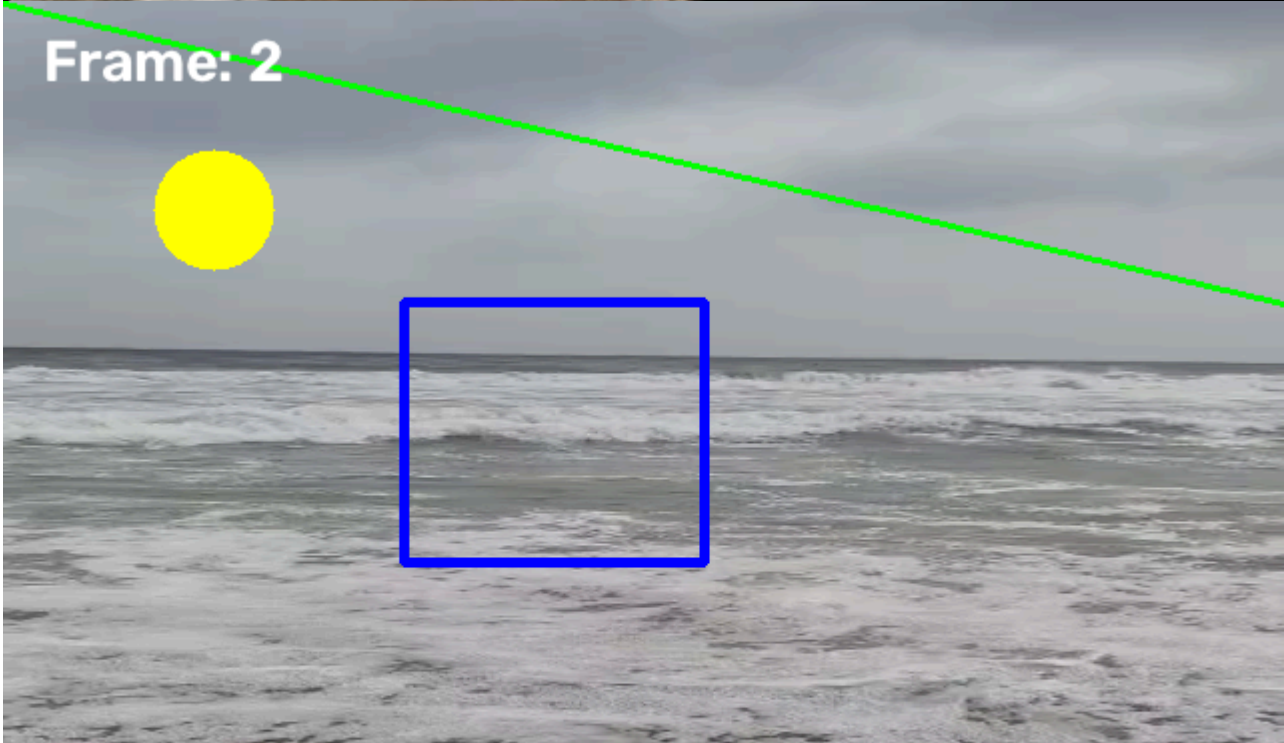
Resizing frames to: 640x480 for consistent drawing and perspective transform.

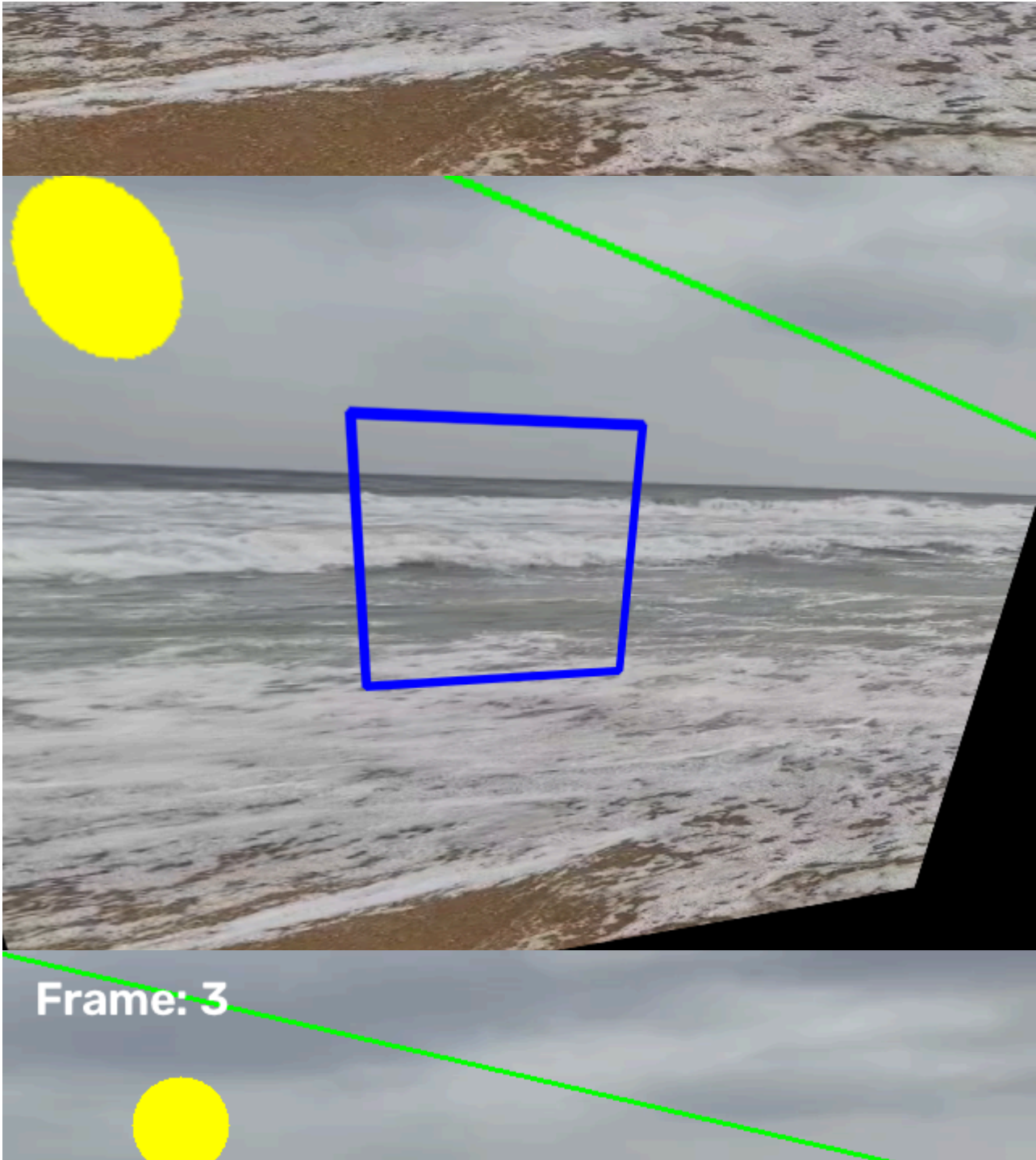
Attempting to process **30** frames (approx 1.0 second(s))



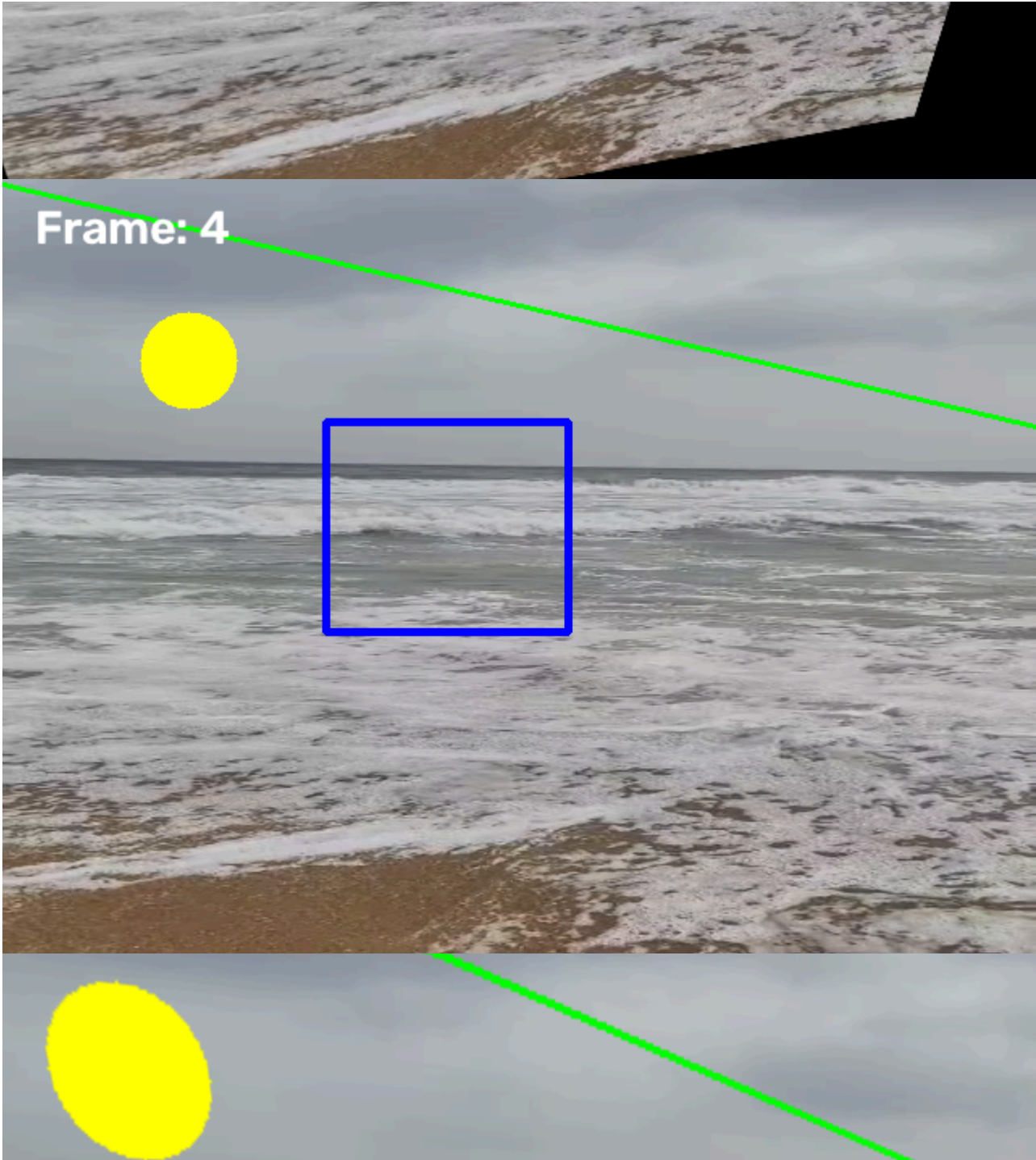


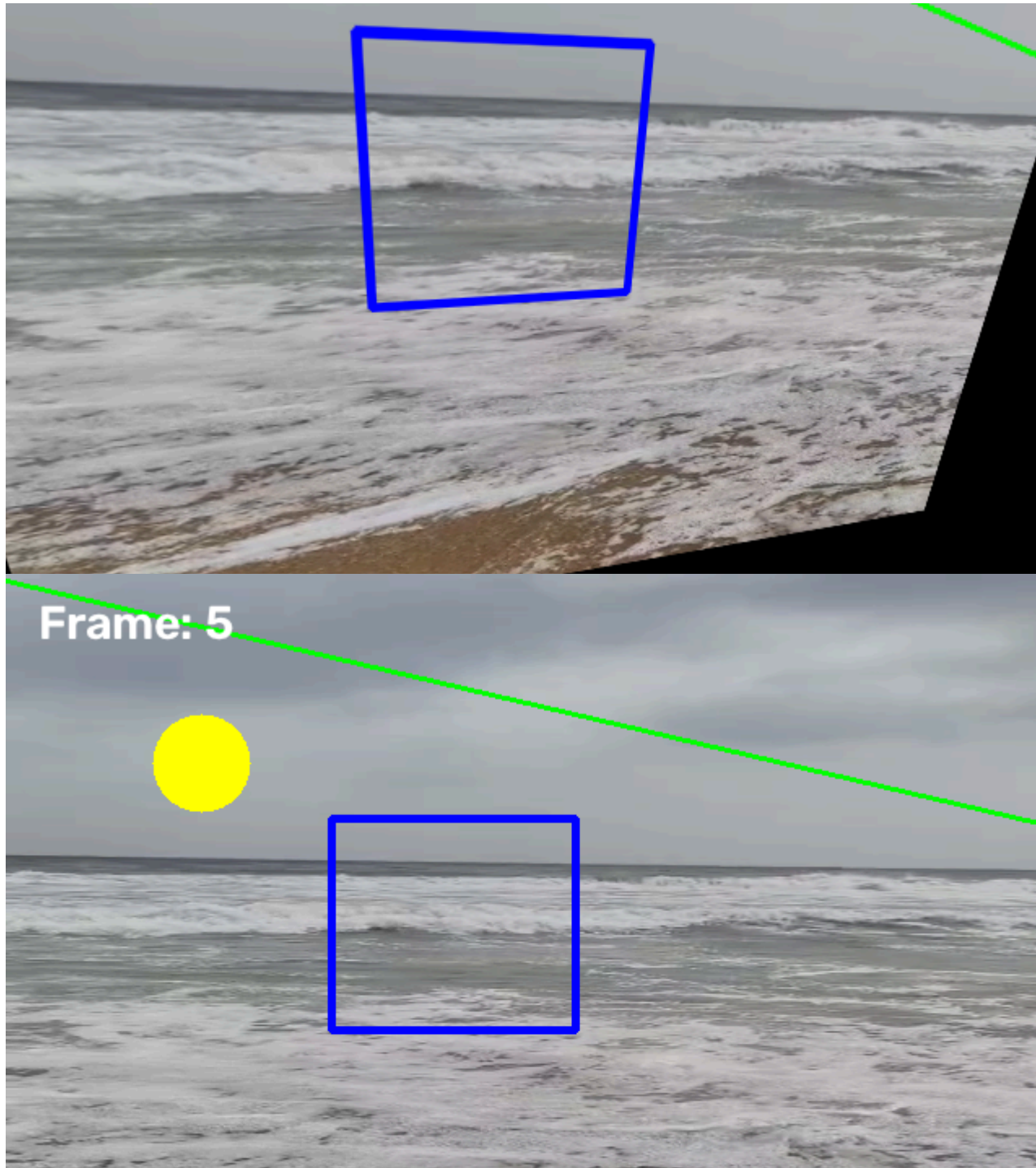
Frame: 2

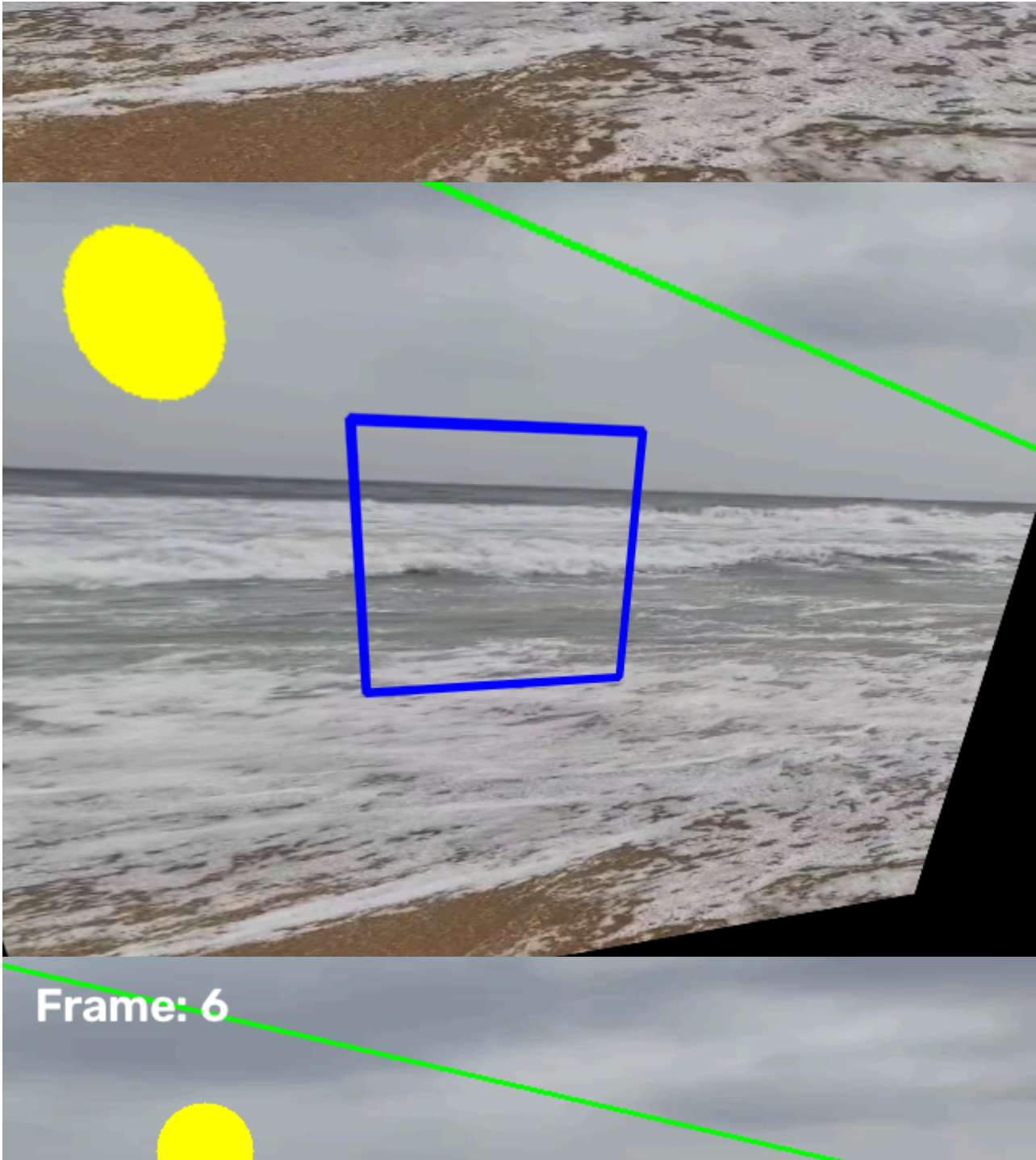


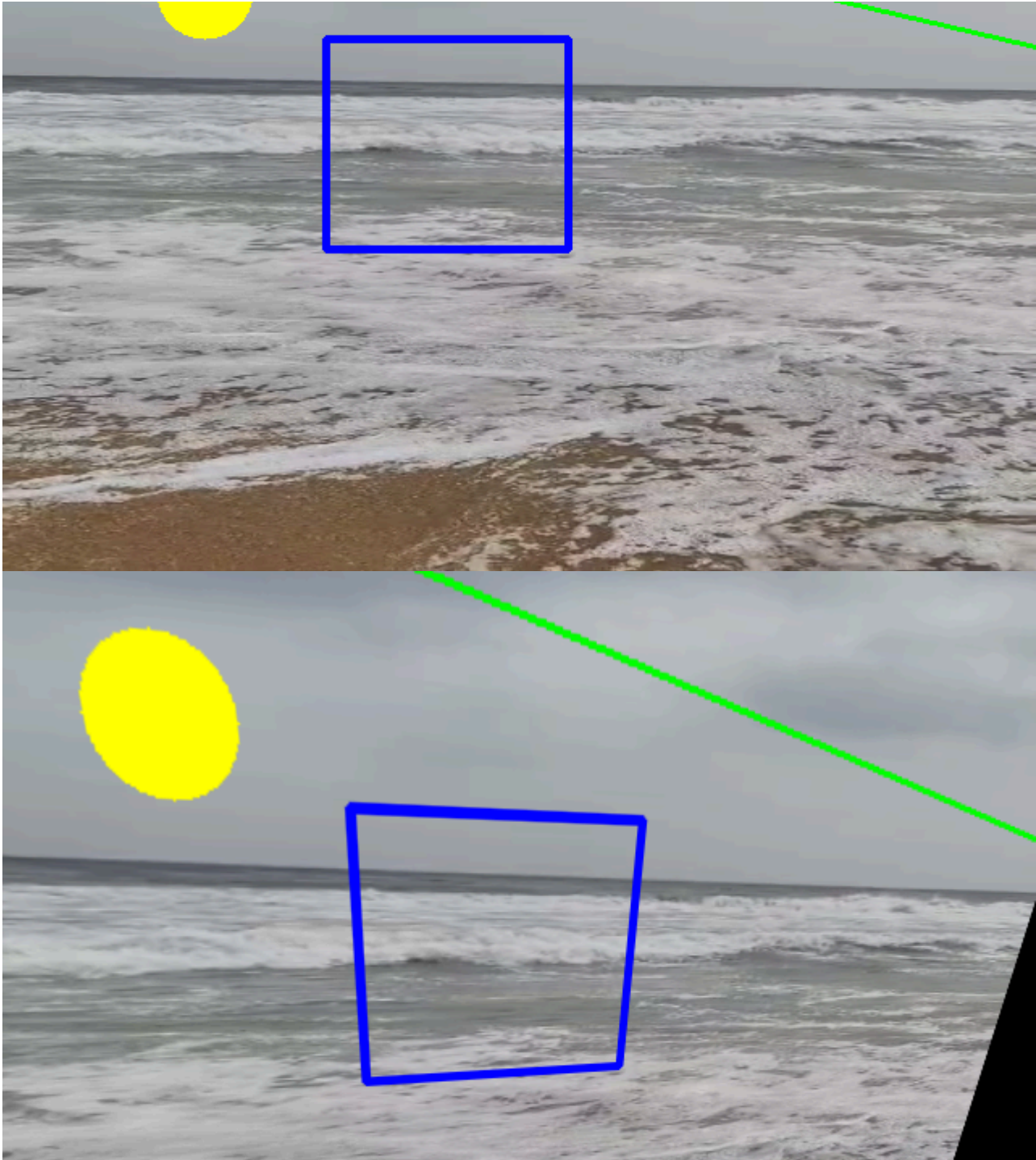


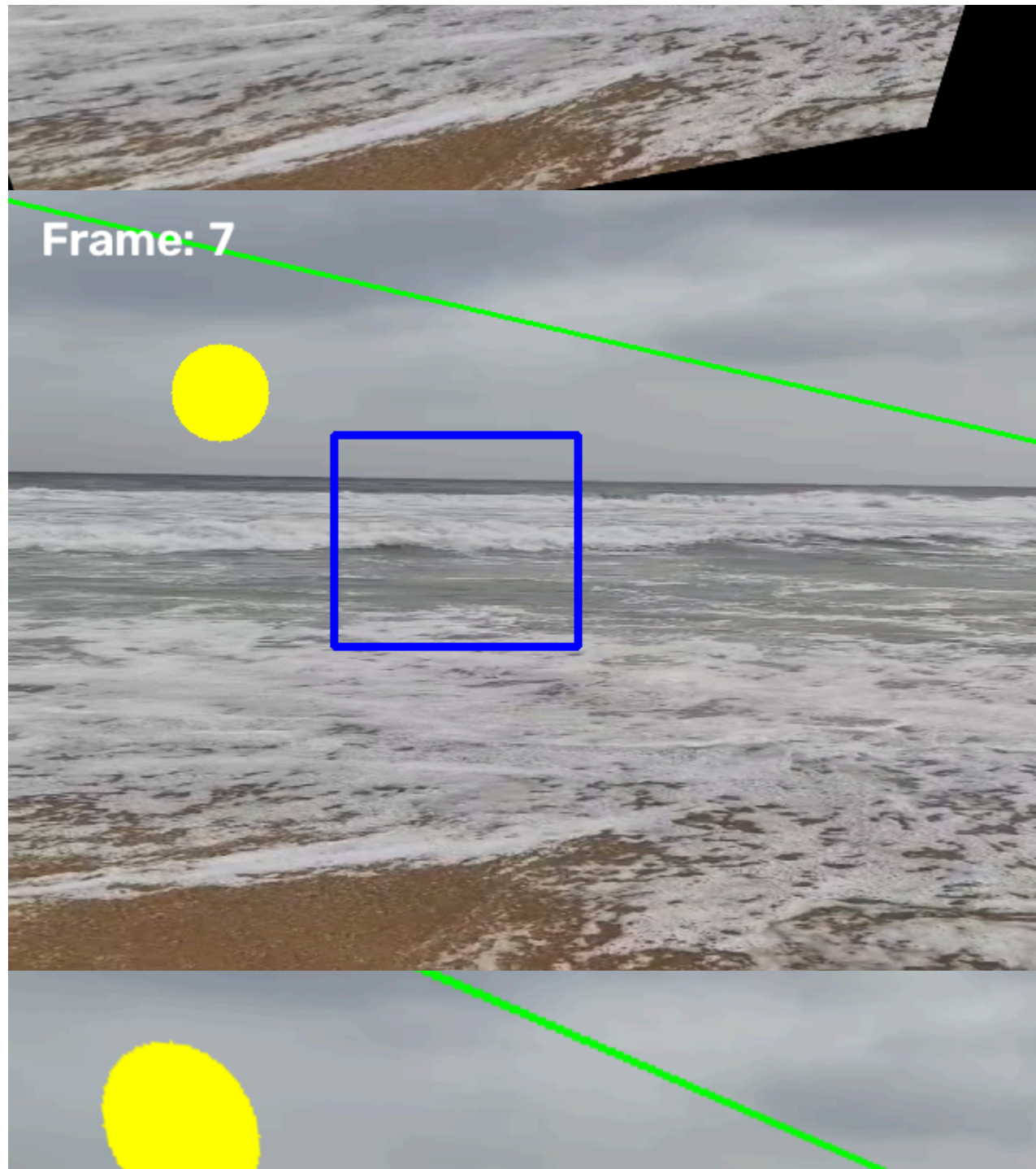


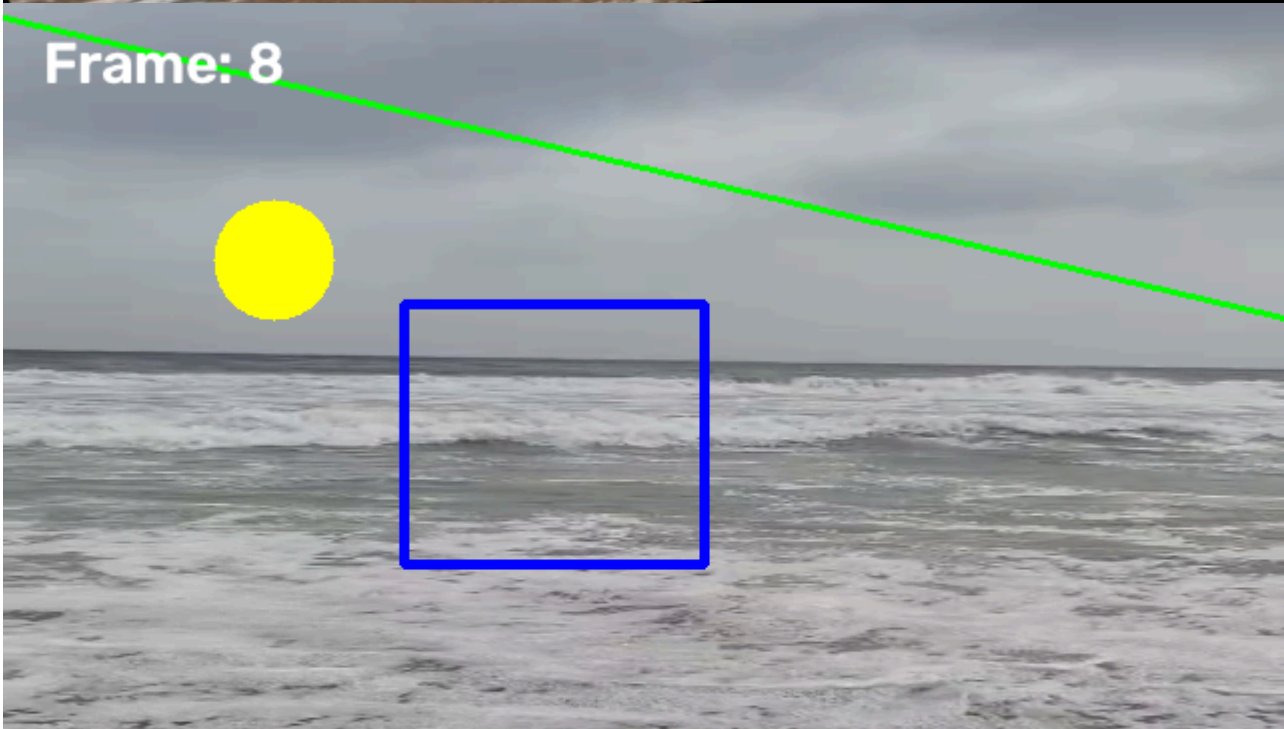


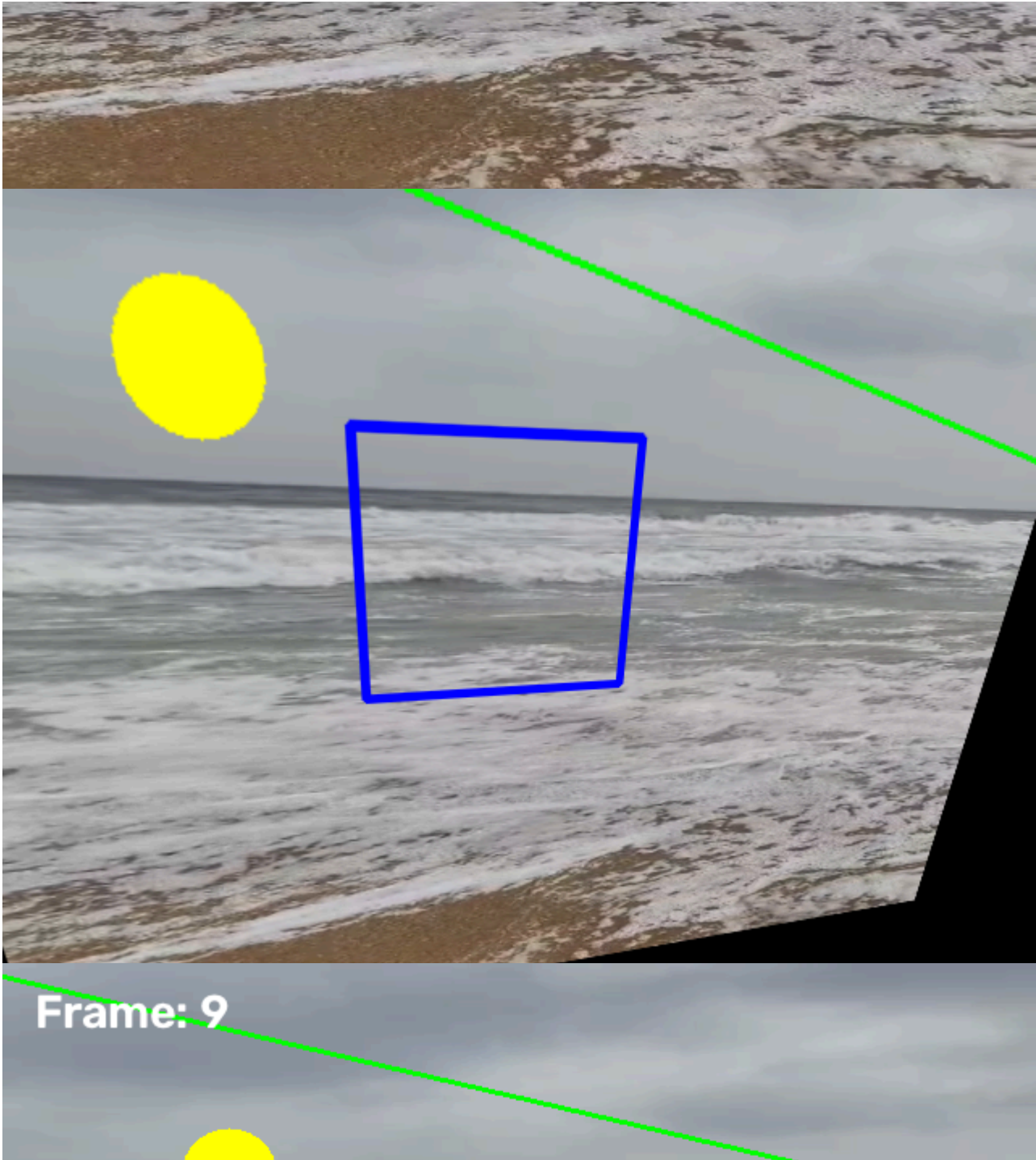


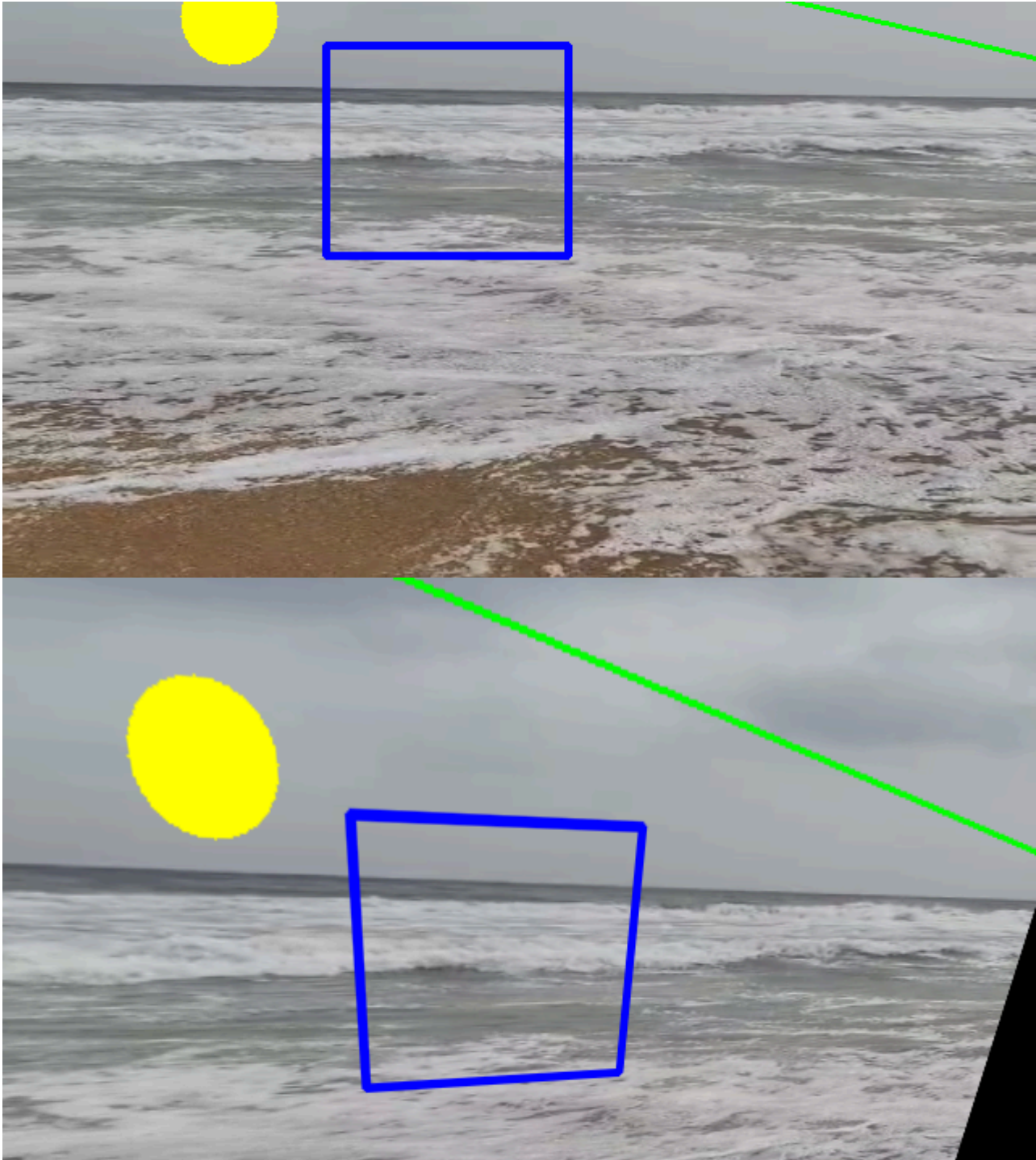


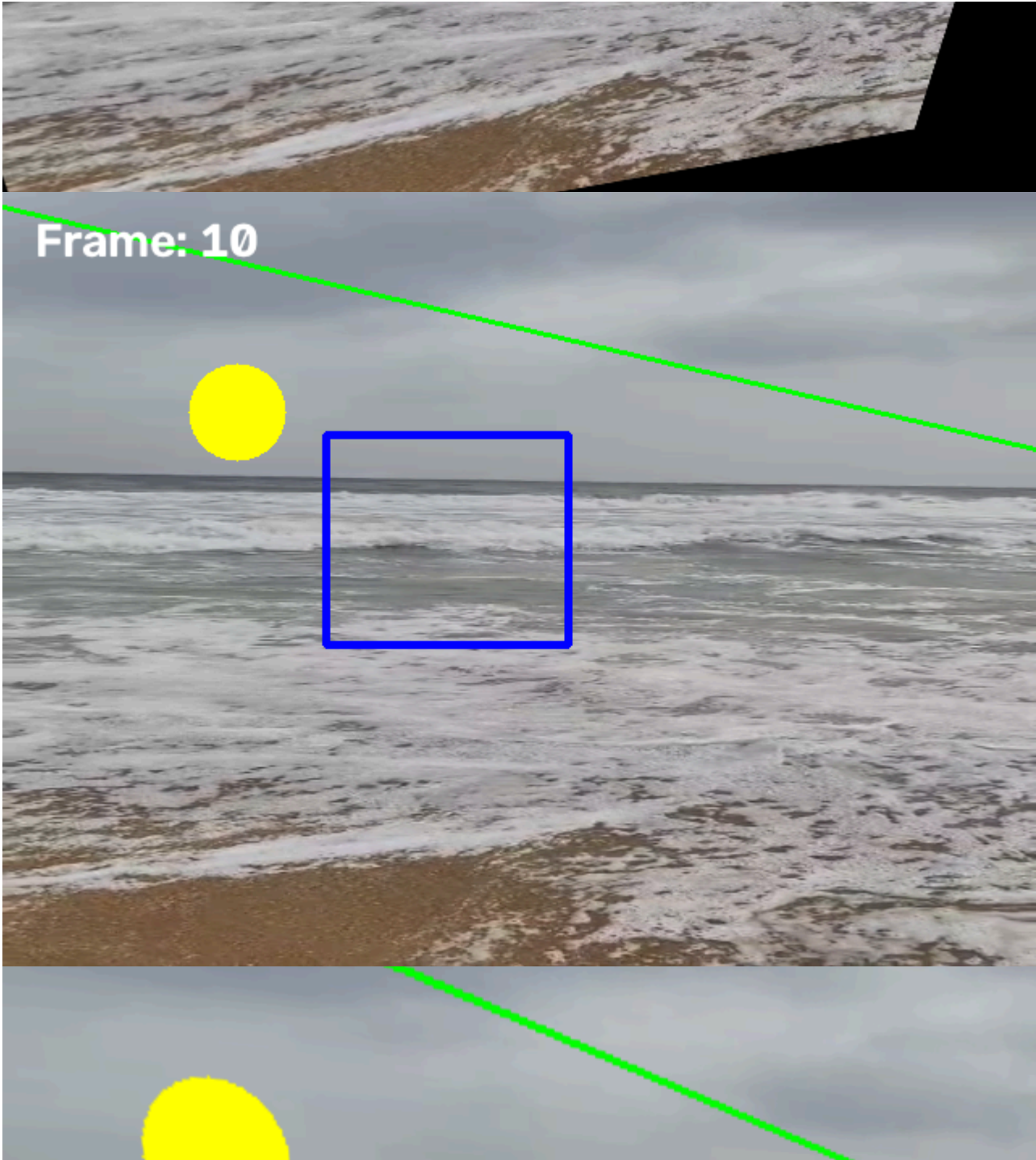






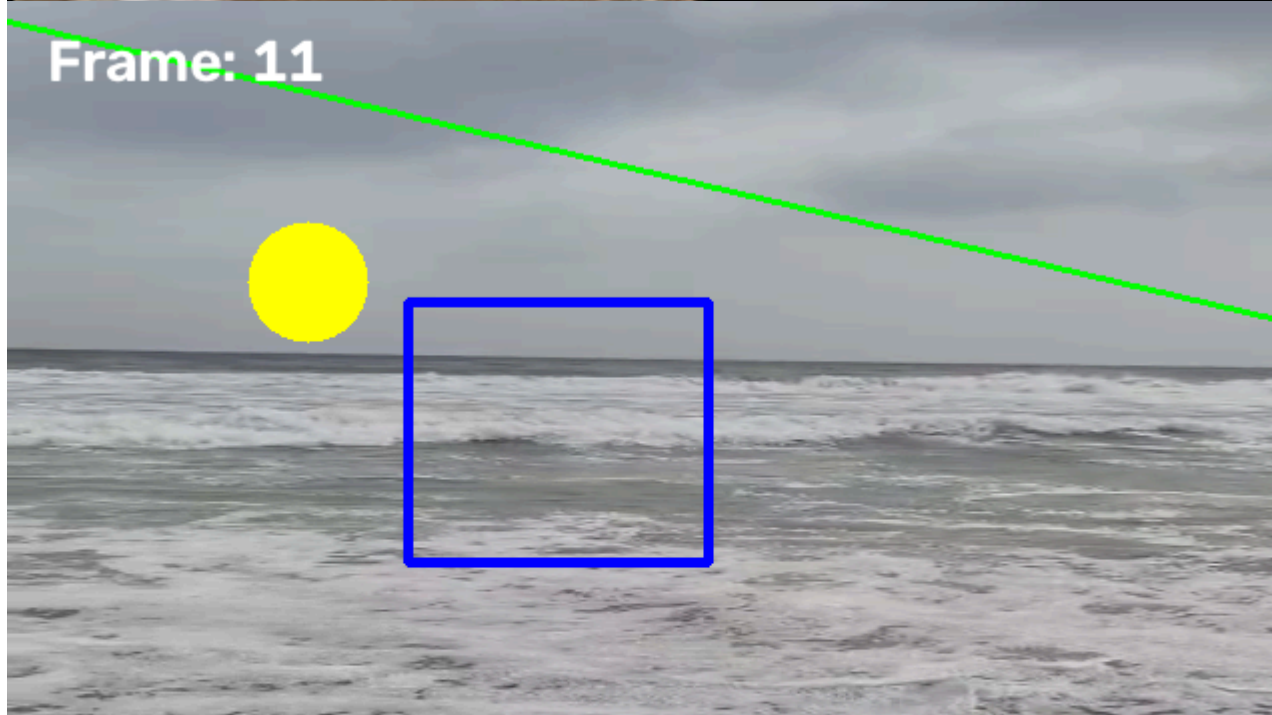


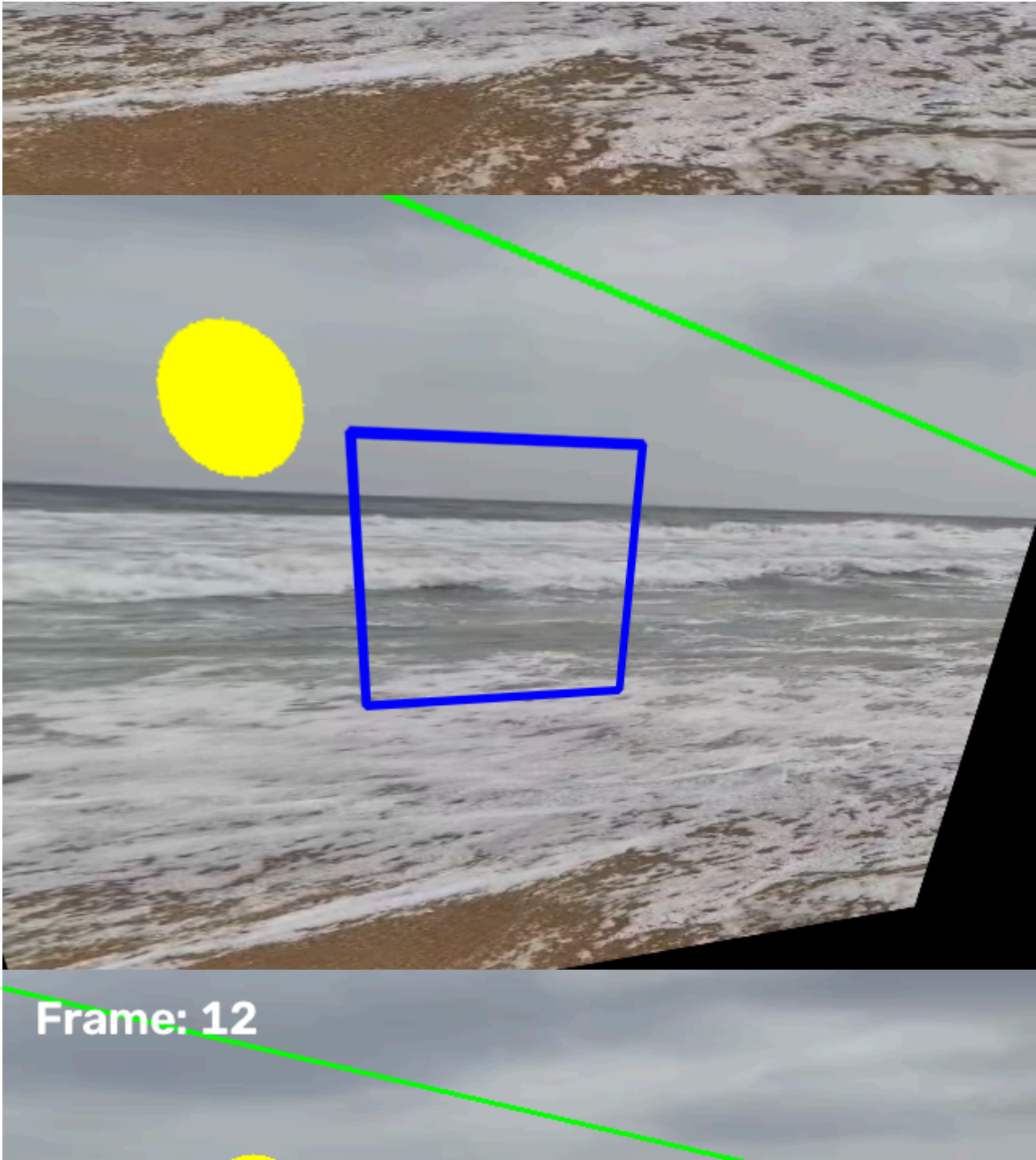


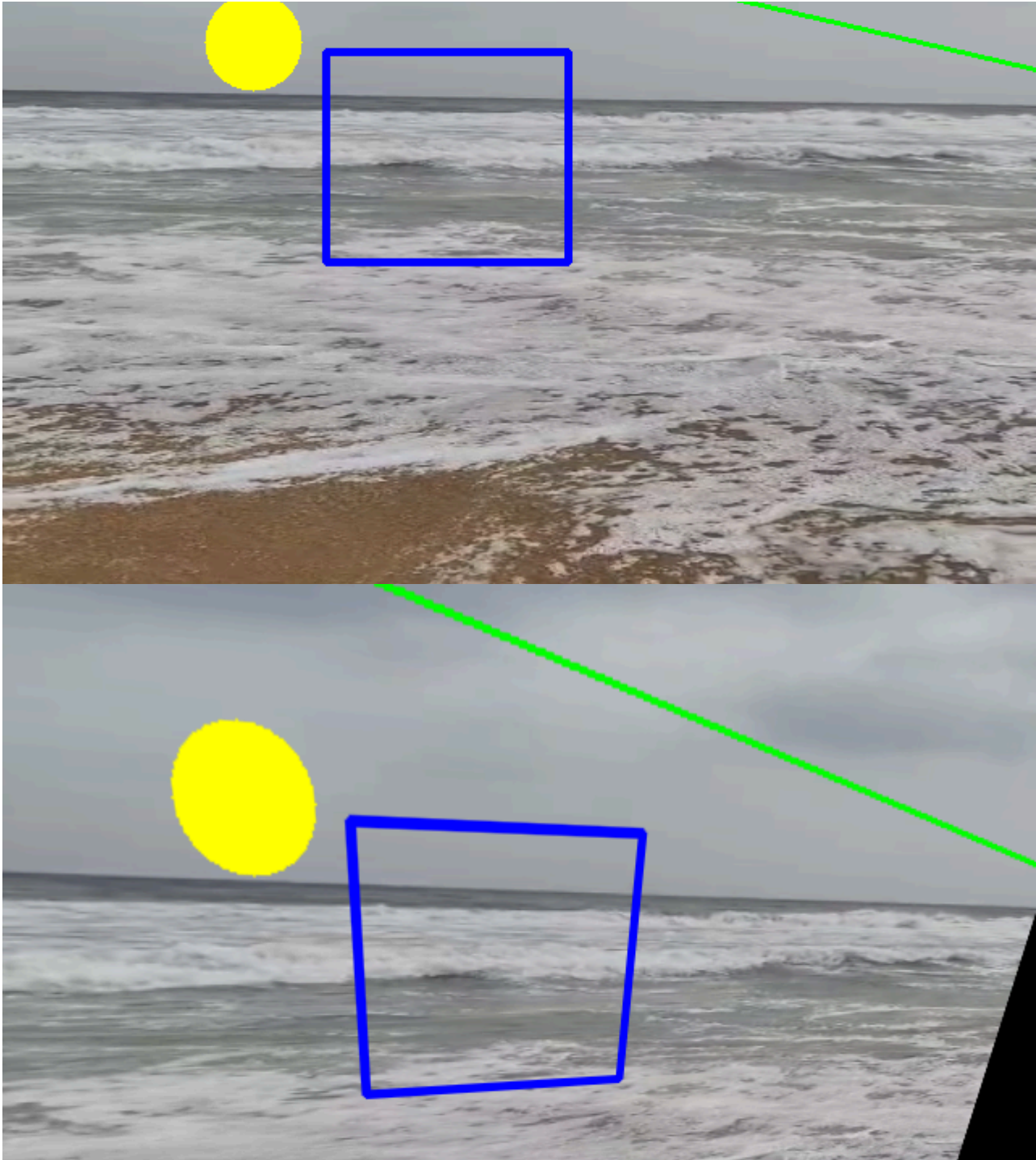


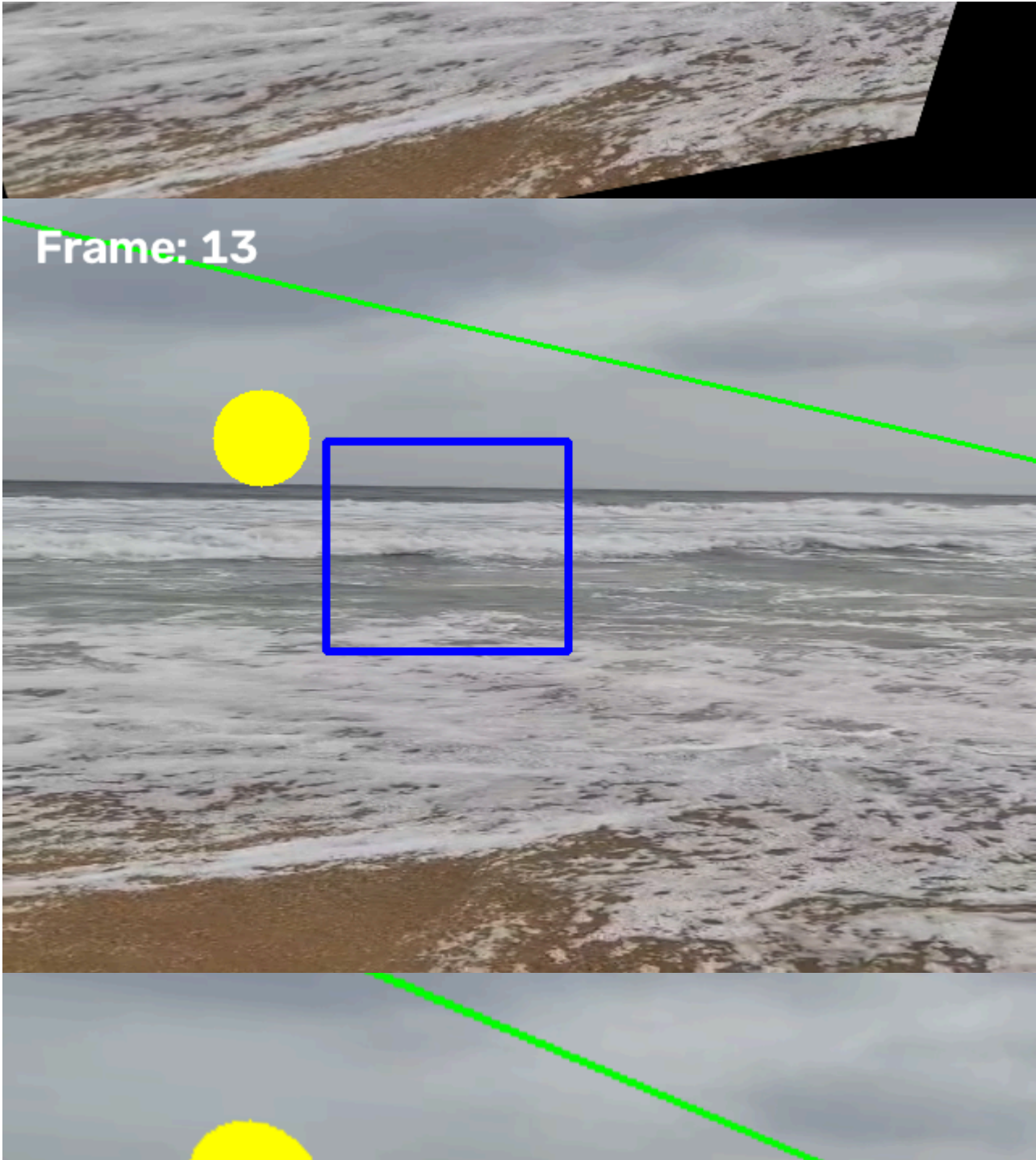


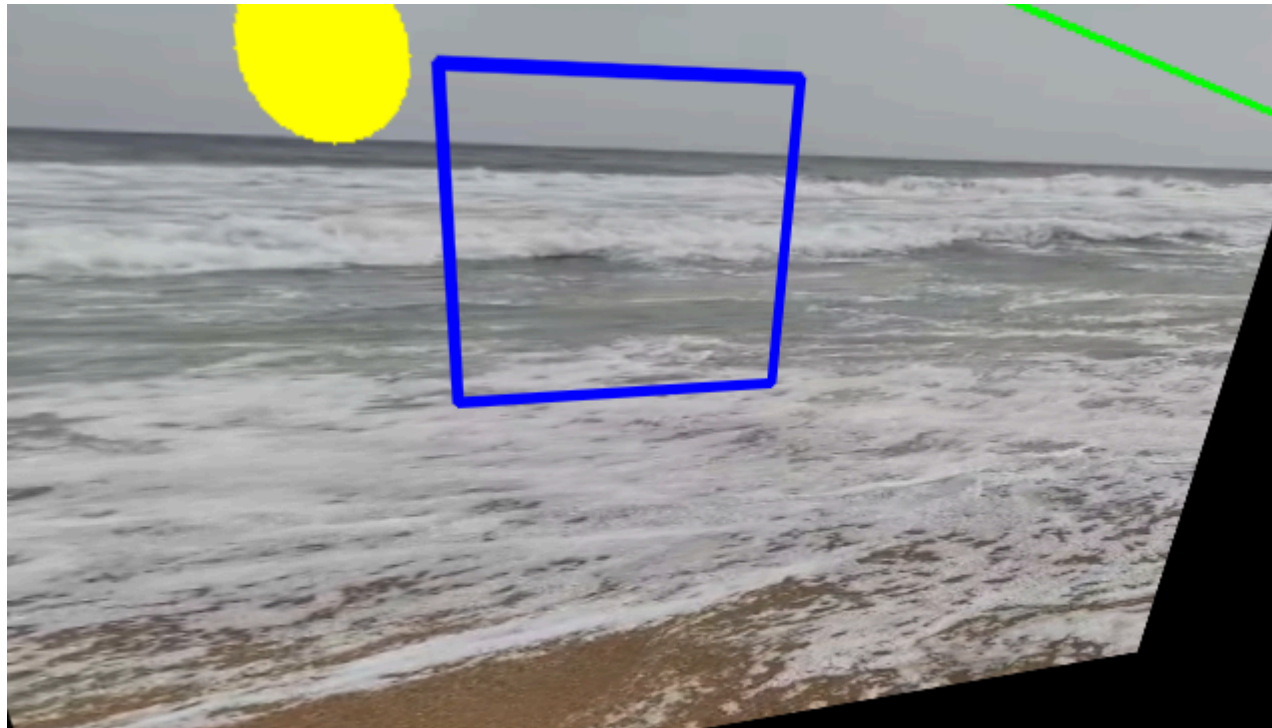
Frame: 11



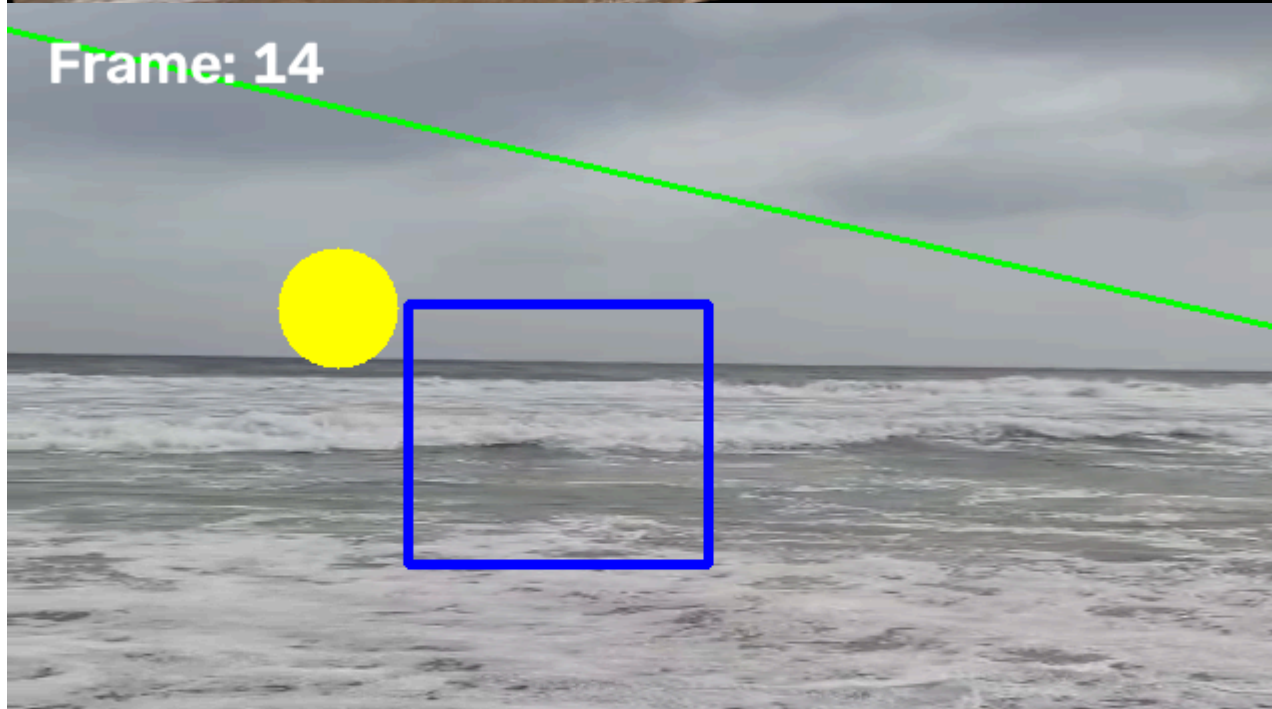


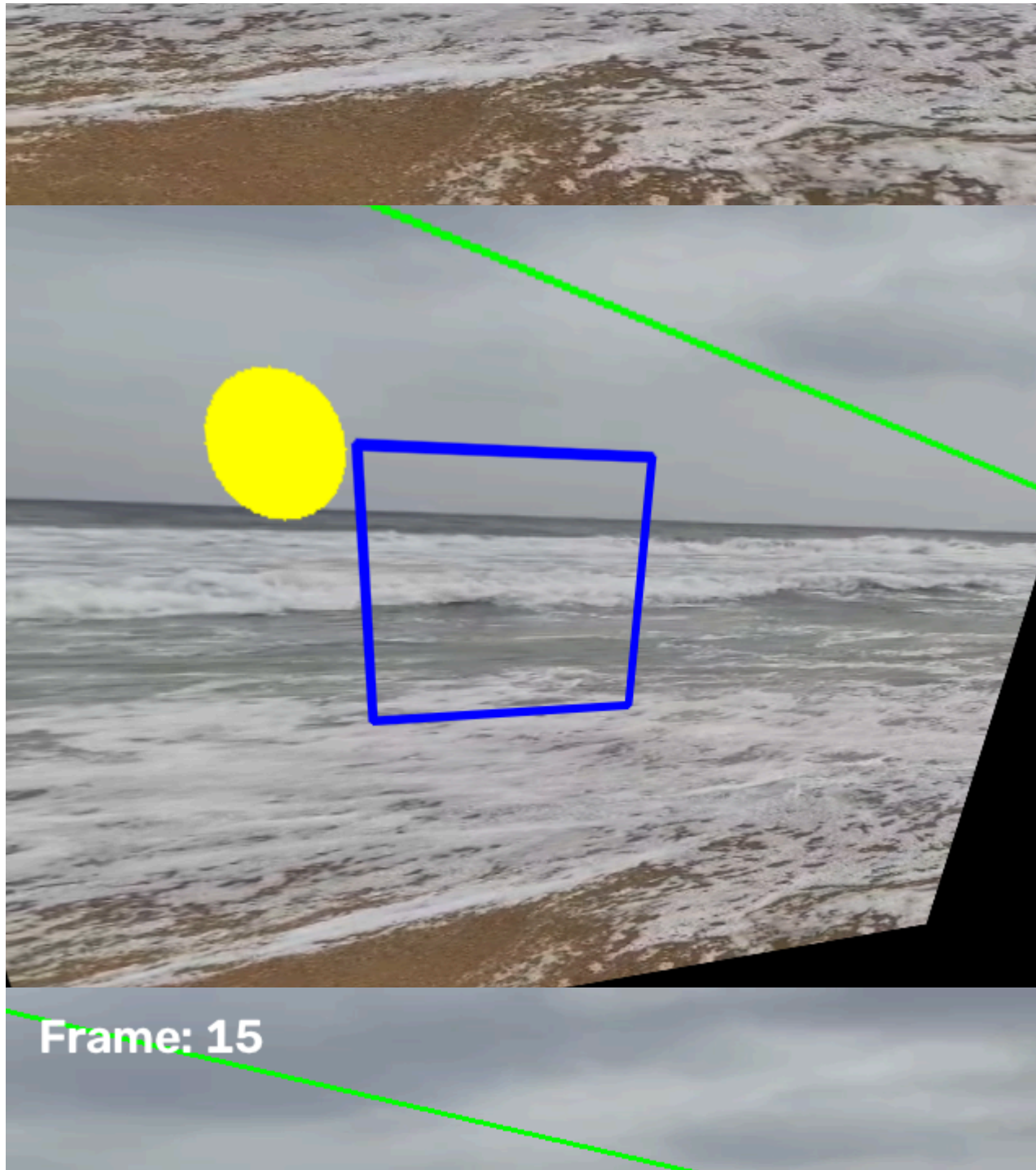




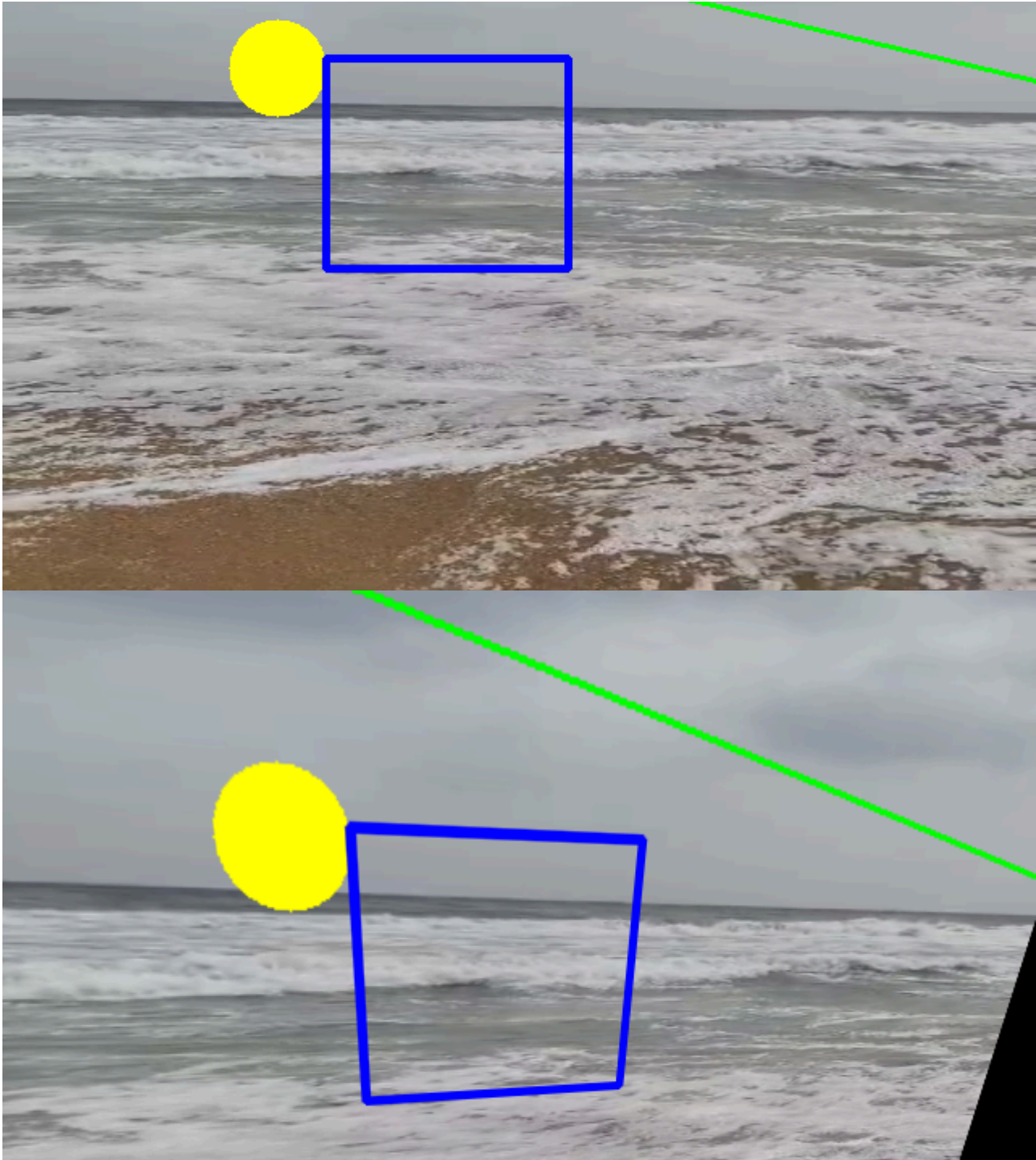


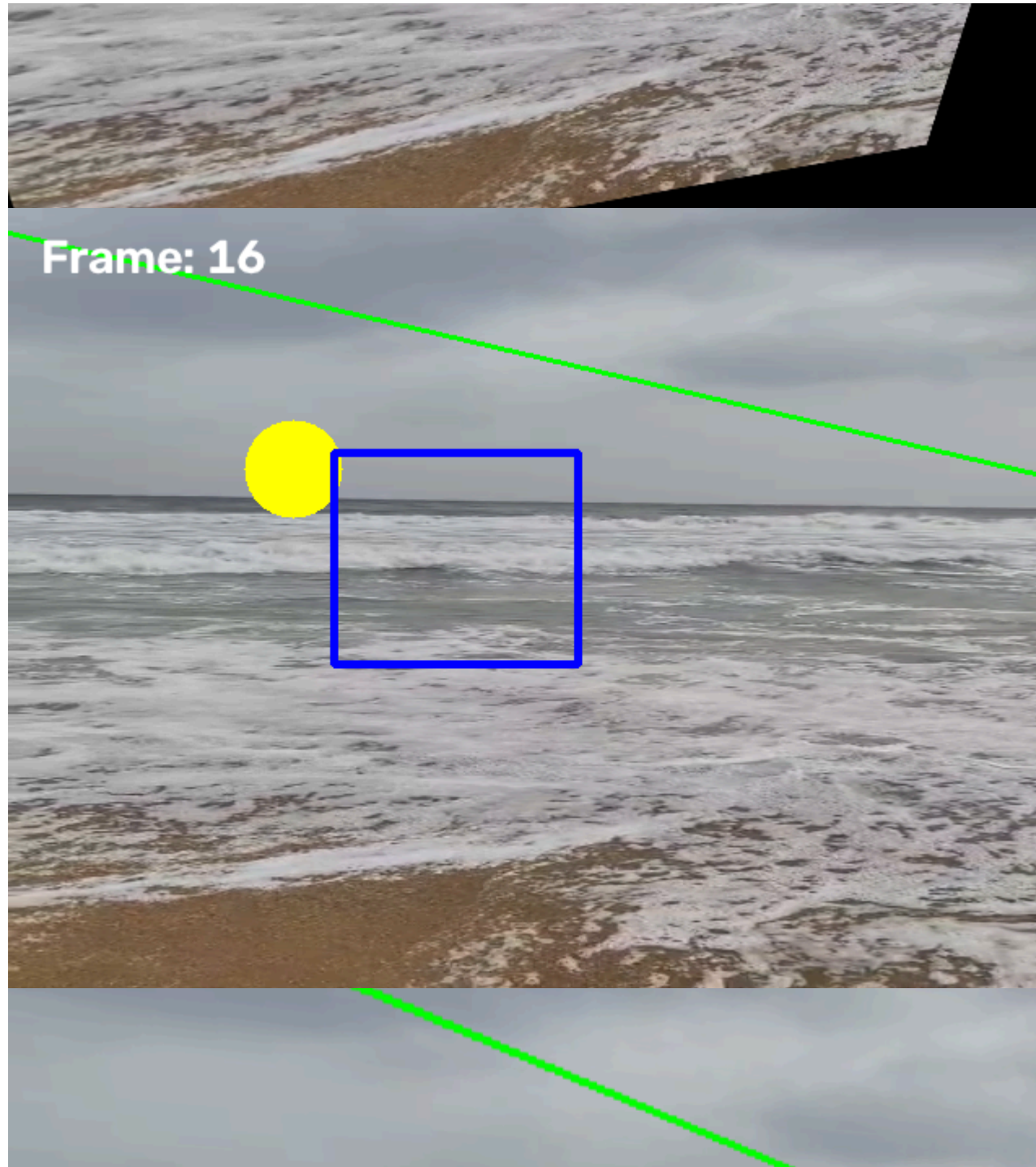
Frame: 14

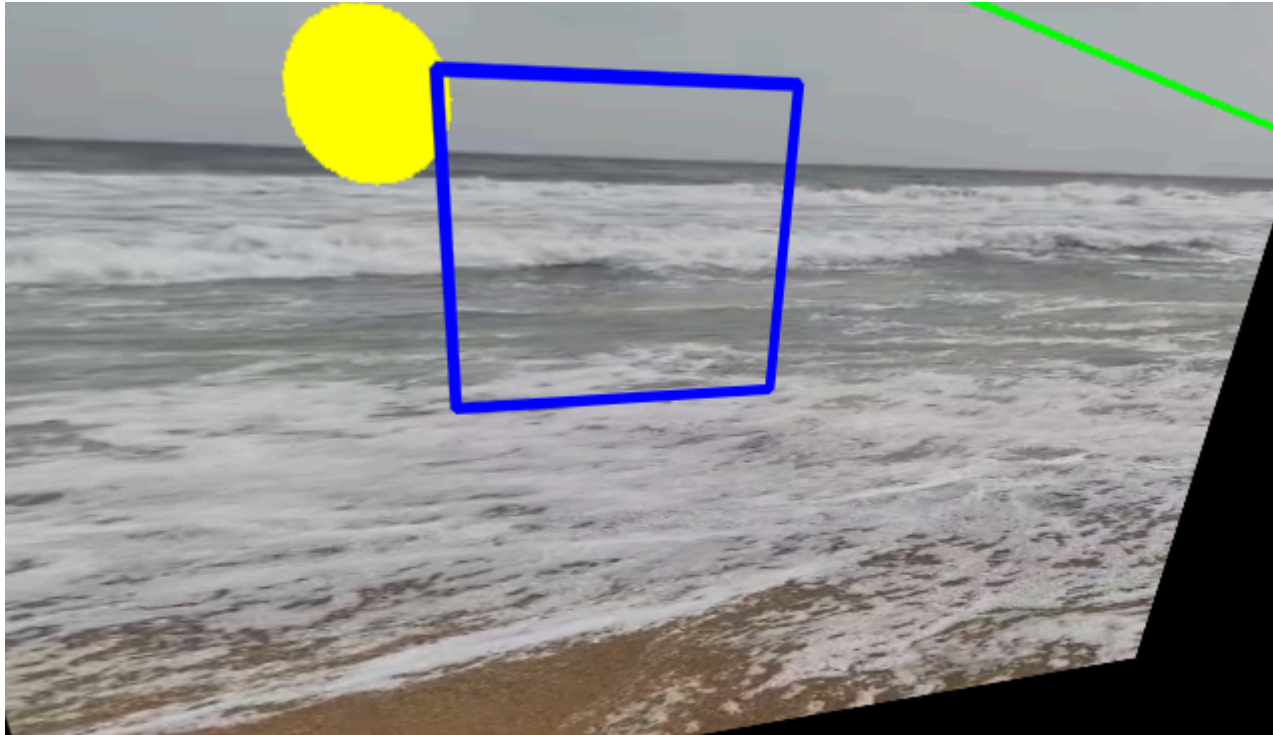




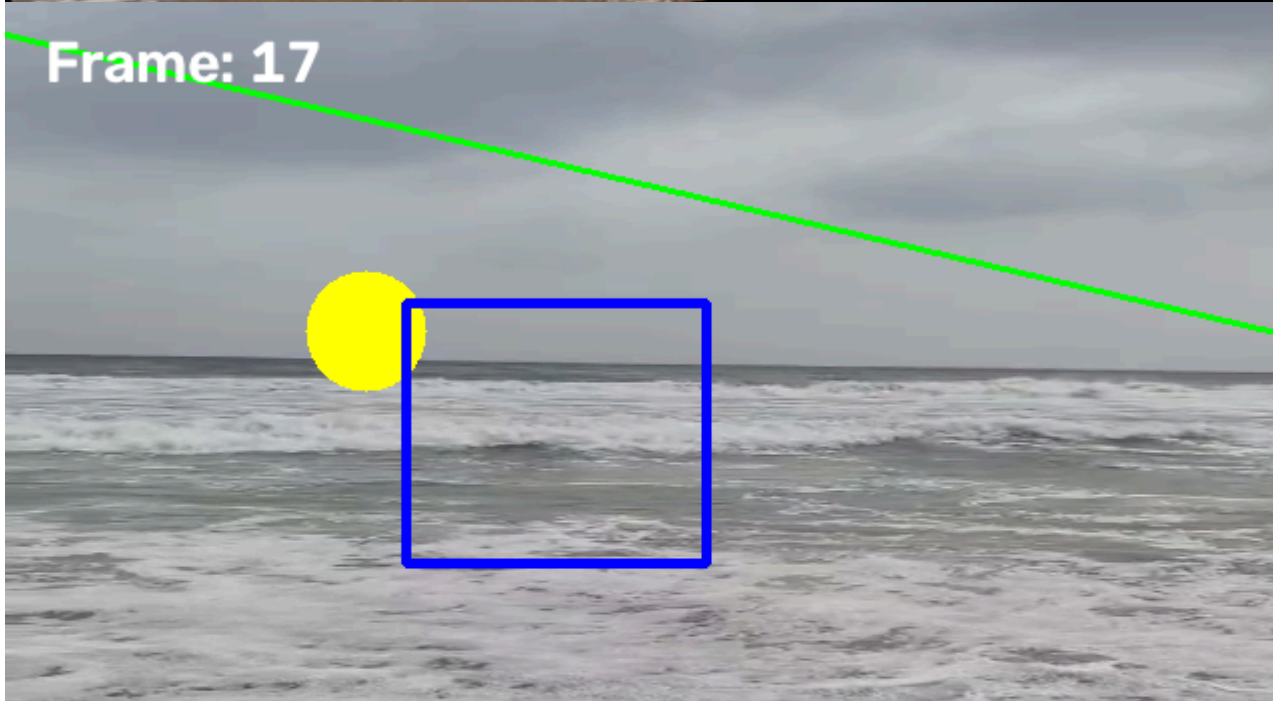
Frame: 15

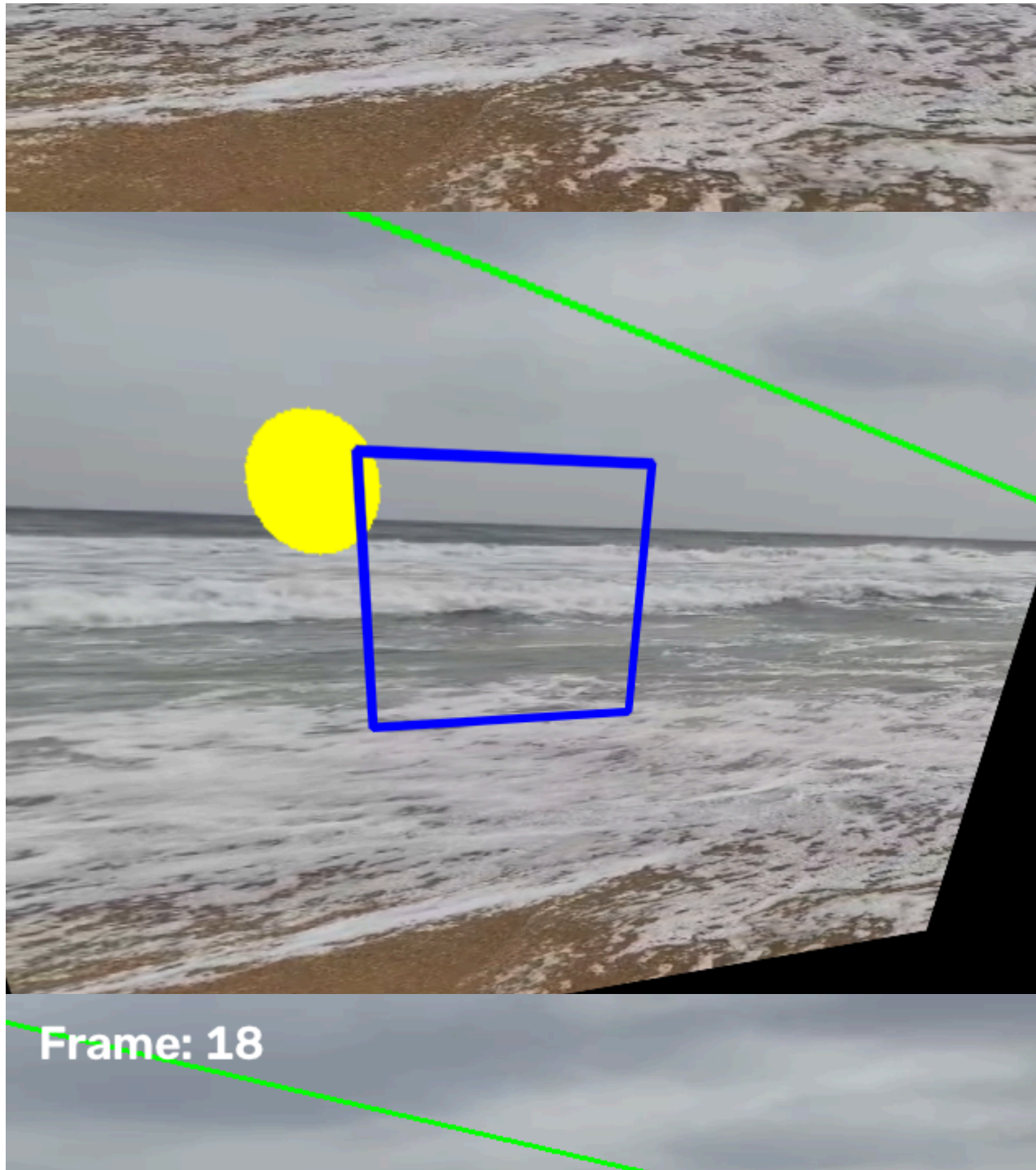






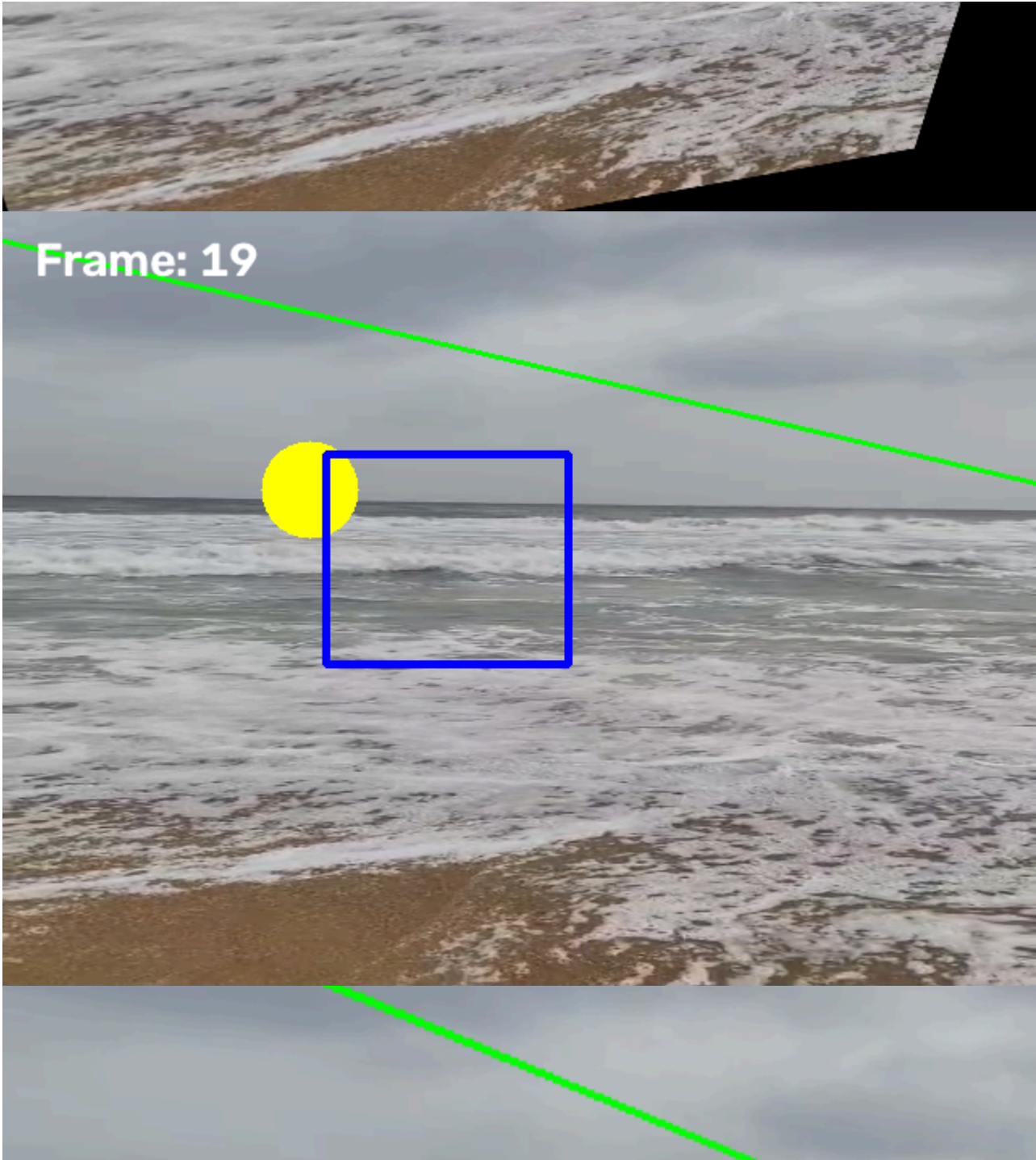
Frame: 17

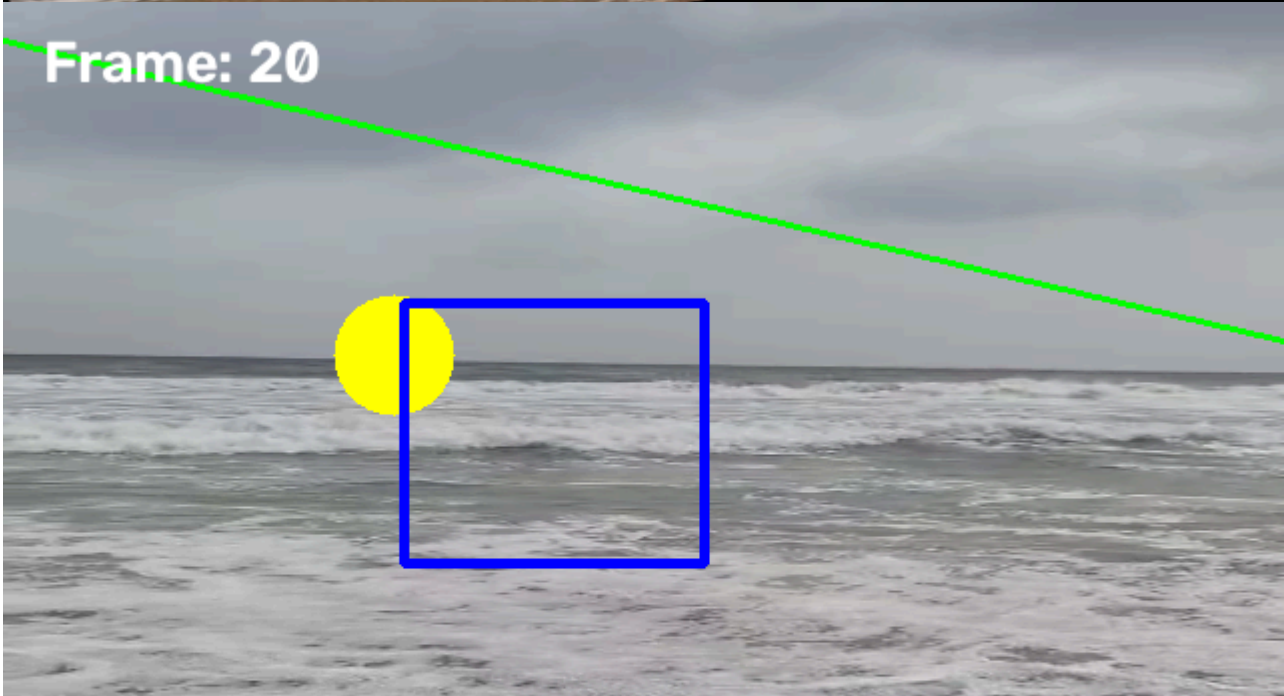
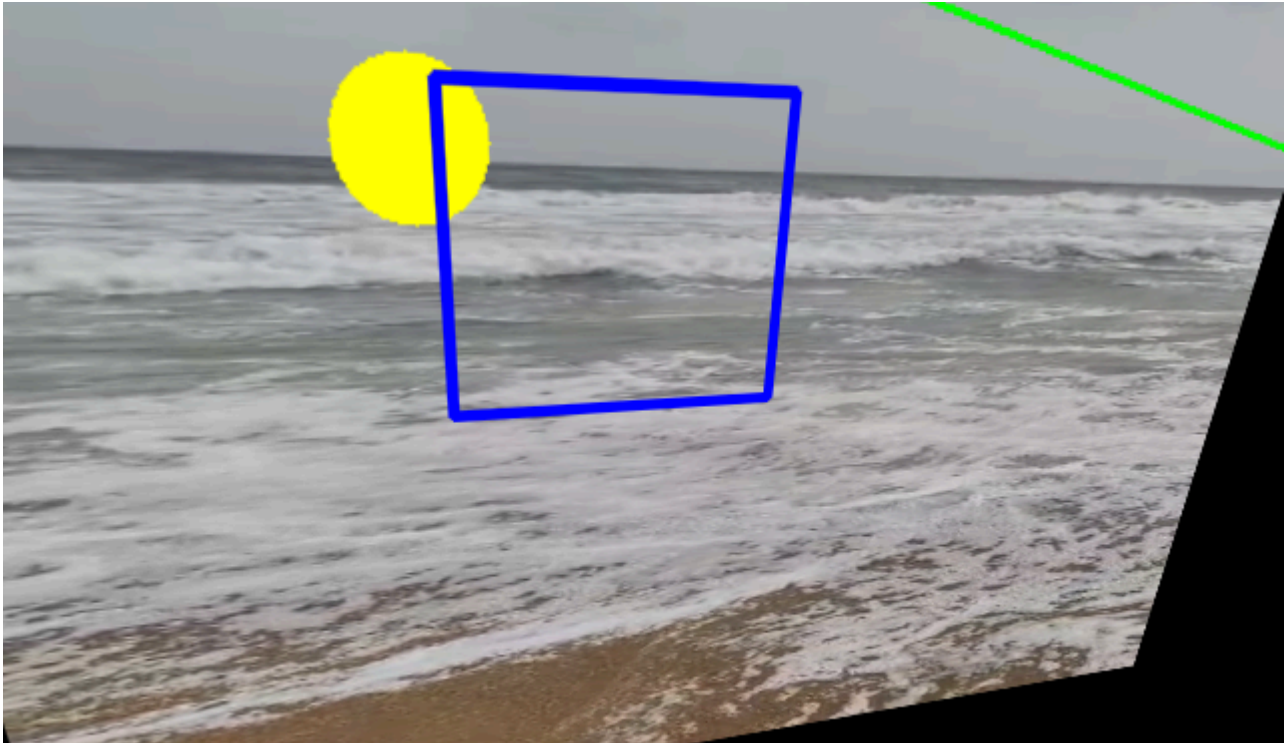


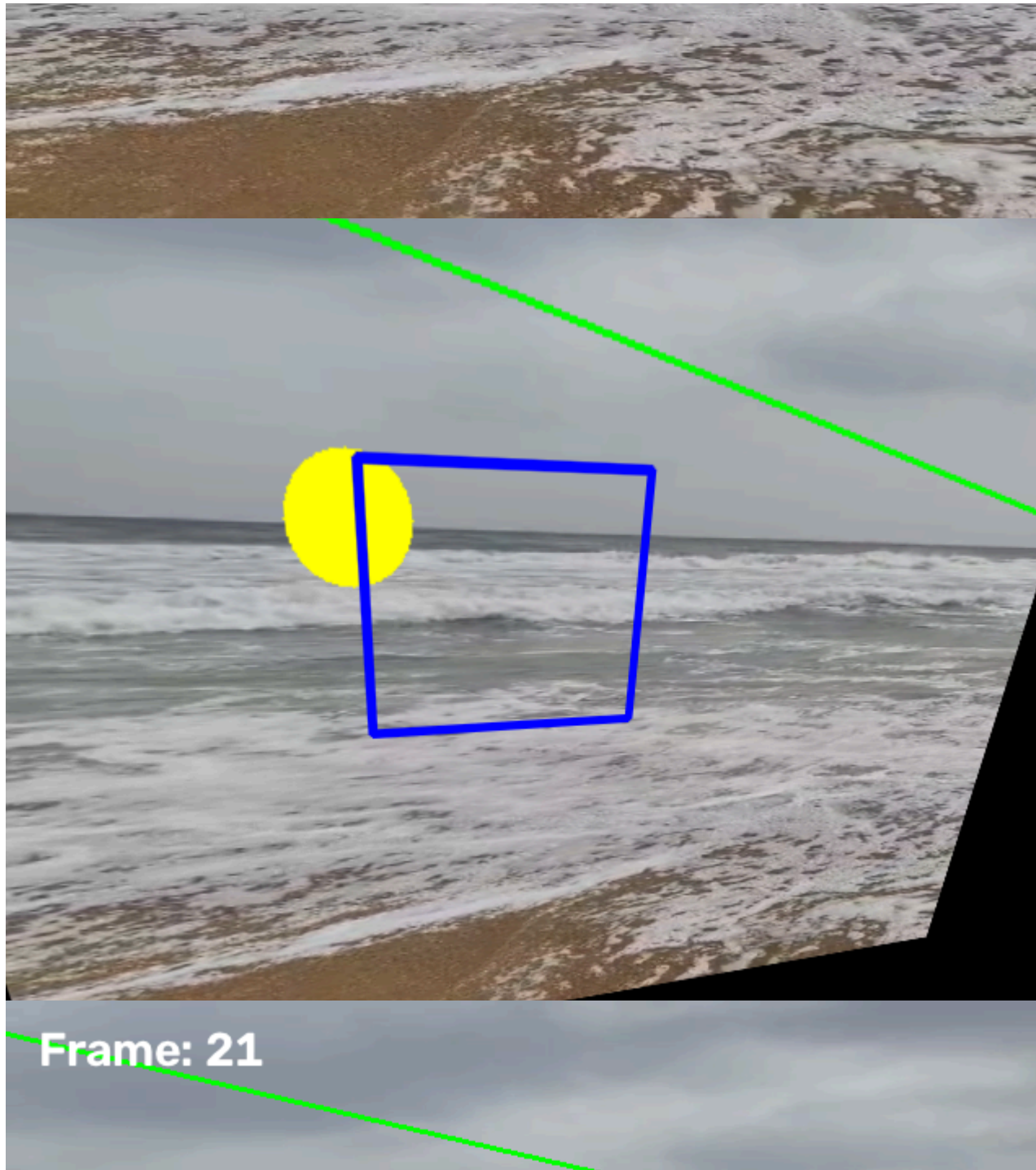


Frame: 18

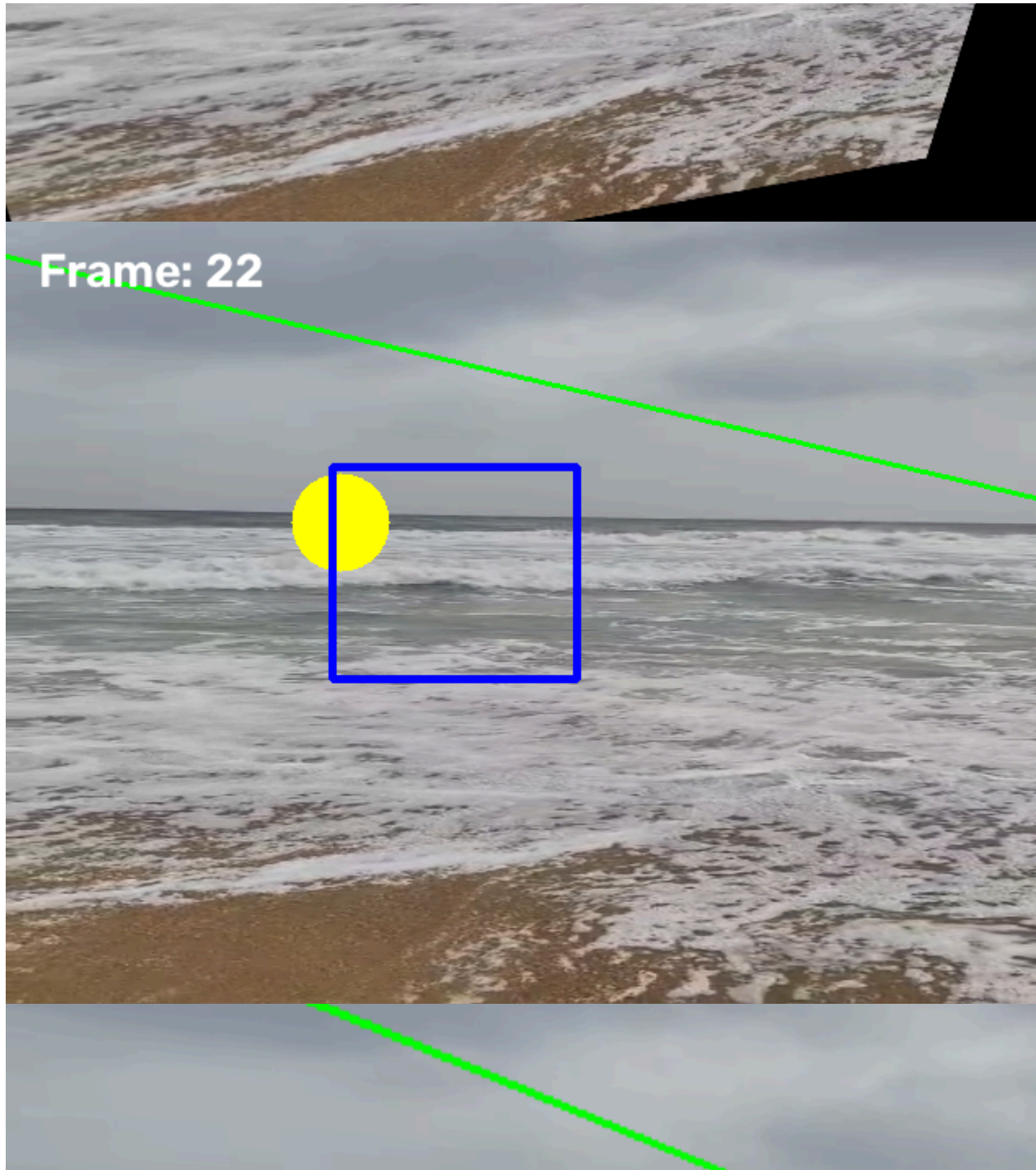


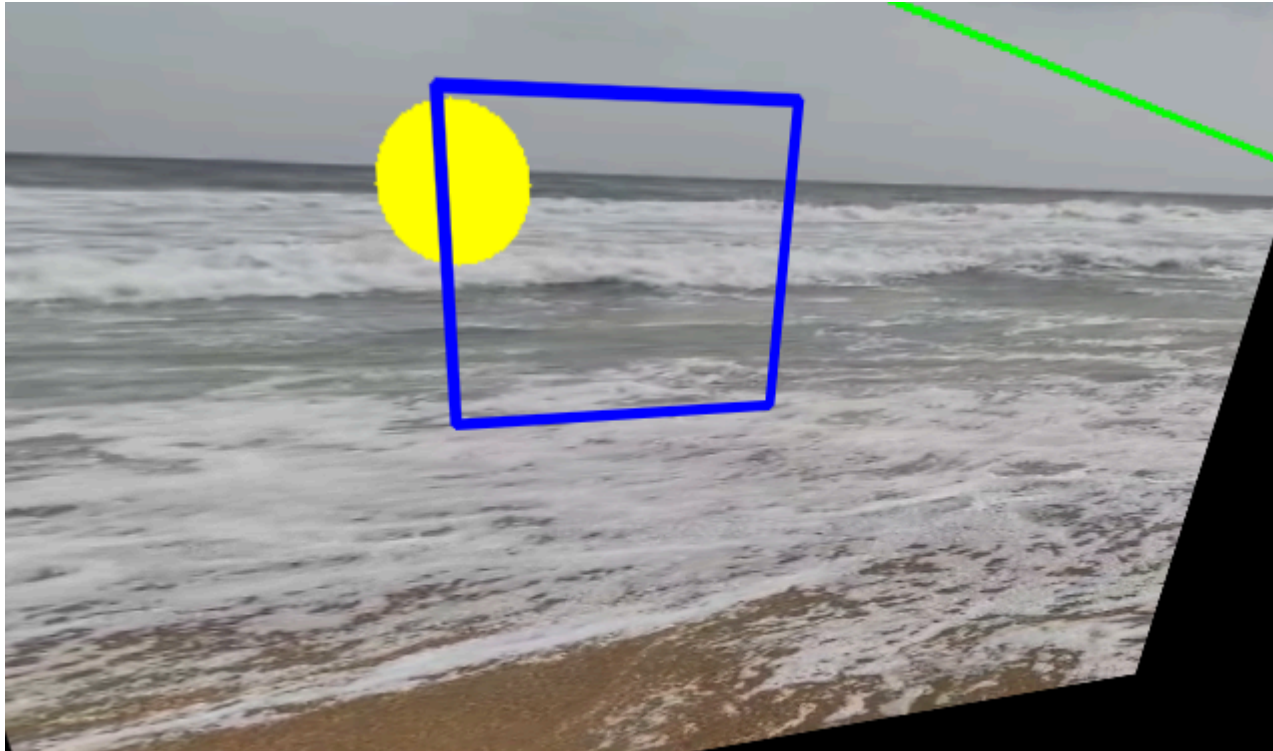




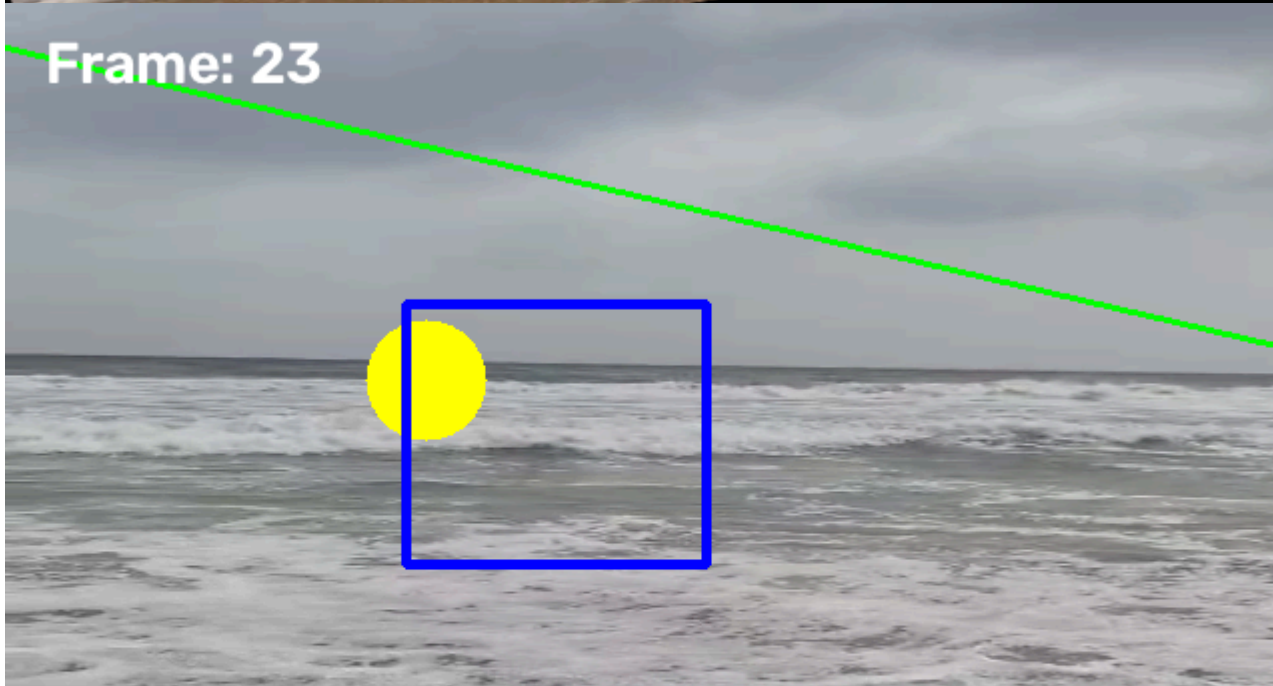


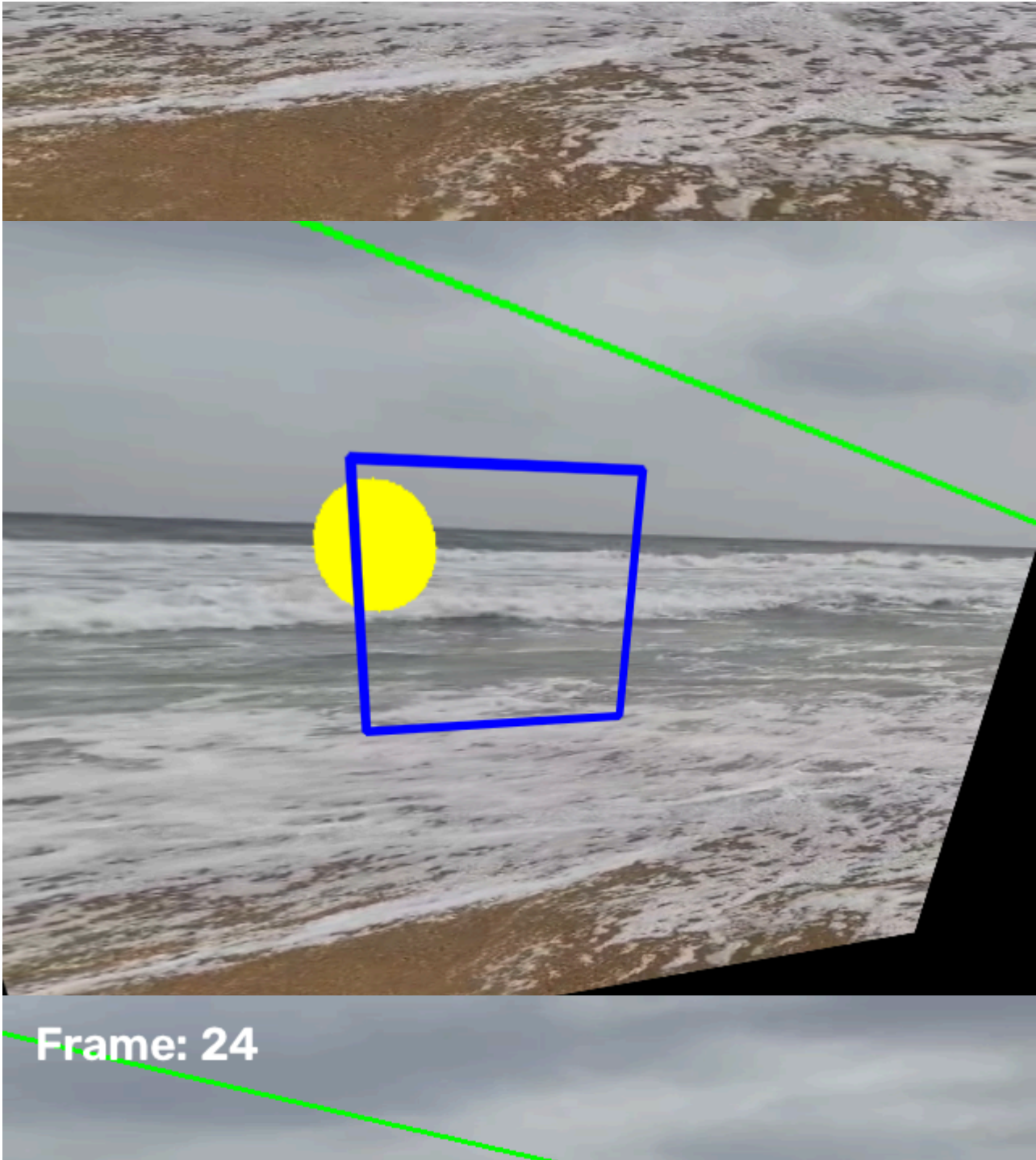




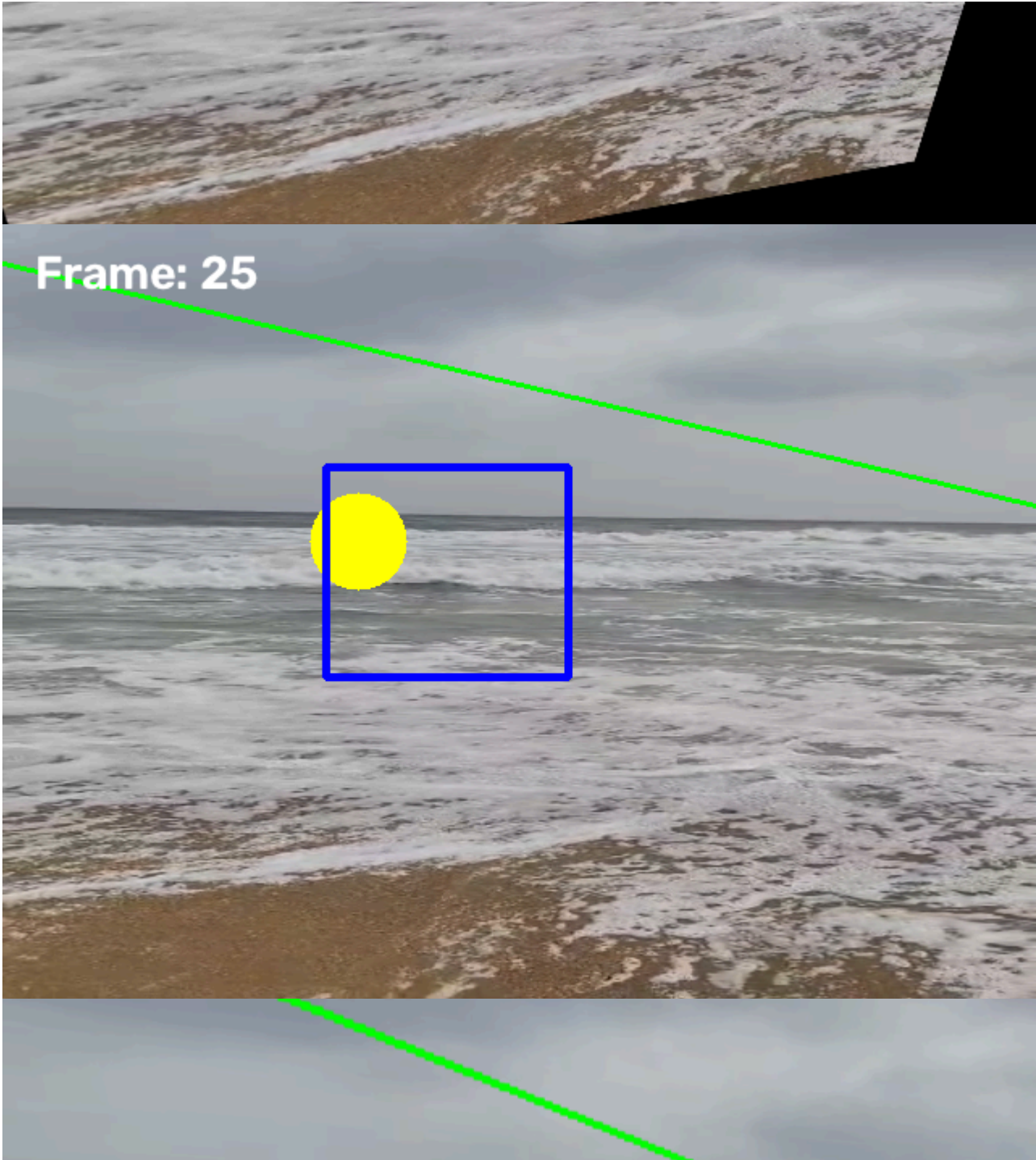


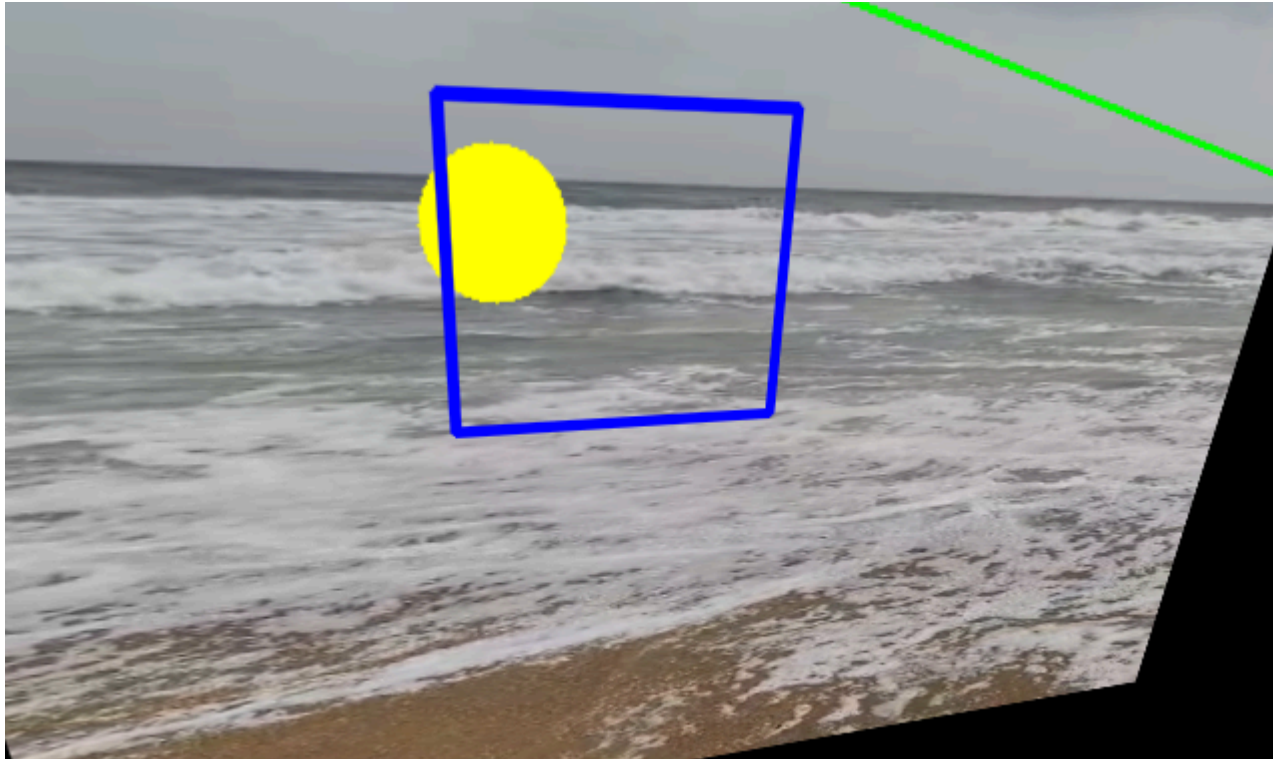
Frame: 23











Frame: 26

